

Universidad Tecnológica de Puebla

Organismo Público Descentralizado del Gobierno del Estado de Puebla

SITIO WEB DE LA ESCUELA DE POSGRADOS DE EXCELENCIA

QUE PARA OBTENER EL TÍTULO DE TÉCNICO SUPERIOR UNIVERSITARIO EN

TECNOLOGÍAS DE LA INFORMACIÓN ÁREA DESARROLLO DE SOFTWARE MULTIPLATAFORMA

PRESENTA:

MARÍA JOSÉ HERNÁNDEZ GUEVARA - 2311081902

ASESORES DE ESTADÍA:

ASESOR EXTERNO

ADOLFO ANGEL MELCHOR AGUIRRE

ASESOR INTERNO

PEDRO MARTÍNEZ GALAVIZ

AGOSTO 2025



ÍNDICE

INTRODUCCIÓN 6
CAPÍTULO I ESCUELA DE POSGRADOS DE EXCELENCIA 7
1.1 MISIÓN 8
1.2 VISIÓN 8
1.3 VALORES 8
1.4 ORGANIGRAMA 9
1.5 PLANTEAMIENTO DEL PROBLEMA 9
1.6 OBJETIVO 11
CAPÍTULO II MARCO TEÓRICO 12
2.1 FUNDAMENTOS BÁSICOS DE INTERNET 13
2.1.1 Internet 13
2.1.2 La Web 14
2.1.3 Páginas y Sitios Web 14
2.1.4 Tipos de sitios Web 15
2.1.5 Navegadores y Herramientas para Desarrolladores 16
2.1.5.1 Microsoft Edge 16
2.1.5.2 Google Chrome 17
2.1.5.3 Firefox 18
2.1.6 Otros Servicios de Internet 19
2.1.6.1 Correo Electrónico 19
2.1.6.2 FTP 20
2.1.6.3 P2P 21
2.1.7 Redes Sociales 21
2.2 HERRAMIENTAS DE PROGRAMACIÓN 23
2.2.1 Lenguajes 23
2.2.1.1 HTML5 23
2.2.1.2 JavaScript 24
2.2.1.3 Ajax 25
2.2.1.4 PHP 26
2.2.1.5 ASP 27

2.2.1.6 JSON 28
2.3 DISEÑO WEB 30
2.3.1 Fundamentos del Diseño Web 30
2.3.1.1 Tipografía 30
2.3.1.2 Color 31
2.3.1.3 Composición 32
2.3.1.4 Otras consideraciones 32
2.3.1.5 Diseño Web Adaptativo 33
2.3.1.6 Diseño Web Sensible 34
2.3.1.7 Importancia de la imagen Web corporativa 34
2.3.2 Diseño de Páginas Web 34
2.3.2.1 HTML 34
2.3.2.2 HTML avanzado 43
2.3.2.3 HTML Dinámico 45
2.3.2.4 JavaScript 47
CAPÍTULO III SITIO WEB DE LA ESCUELA DE POSGRADOS DE EXCELENCIA 53
3.1. CONFIGURACION DEL ENTORNO DE DESARROLLO 54
3.2. DESARROLLO DEL FRONTEND (Vista del sitio web) 55
3.2.1 Estructura del Documento HTML 55
3.2.2 Estilización con CSS 58
3.2.3 Comportamiento con JavaScript 61
3.3 GESTIÓN DEL CONTENIDO ARCHIVO JSON 64
3.4 DESARROLLO DEL PANEL DE ADMINISTRACIÓN (Back-End) 67
3.4.1 Objetivo del Panel 67
3.4.2 Estructura General 67
3.4.3 Flujo de Funcionamiento 68
3.4.4 Validaciones Implementadas 69
3.4.5 Seguridad 69
3.4.6 Beneficios de la Implementación 69
CONCLUSIÓN 70
GLOSARIO 71
BIBLIOGRAFÍAS 74

ÍNDICE DE IMÁGENES

FIGURA 1. 1 DISEÑO Y ESTRUCTURA DE LA PÁGINA WEB DEL CAMPUS PUEBLA 10
FIGURA 1. 2 DISEÑO Y ESTRUCTURA DEL NUEVO SITIO WEB PARA TODOS LOS CAMPUS 11
FIGURA 2. 1 PRIMERA CONEXIÓN ENTRE ORDENADORES QUE SE DENOMINÓ ARPANET 13
FIGURA 2. 2 TIM BERNERS-LEE CREADOR DE LA WEB 14
FIGURA 2. 3 DIFERENCIA ENTRE SITIO WEB Y PÁGINA WEB 15
FIGURA 2. 4 NAVEGADOR INTERNET EXPLORER 17
FIGURA 2. 5 NAVEGADOR GOOGLE CHROME 18
FIGURA 2. 6 NAVEGADOR FIREFOX 19
FIGURA 2. 7 CORREO ELECTRÓNICO COMO UN SERVICIO DE INTERNET 19
FIGURA 2. 8 ESTRUCTURA GENÉRICA DE UN CORREO ELECTRÓNICO 20
FIGURA 2. 9 P2P REDES LLAMADAS DE PARES, ENTRE IGUALES O PUNTO A PUNTO 21
FIGURA 2. 10 HISTORIA DE LAS REDES SOCIALES 22
FIGURA 2. 11 EJEMPLOS DE TIPOGRAFÍA 30
FIGURA 2. 12 OTRO EJEMPLO DE TIPOGRAFÍAS 30
FIGURA 2. 13 EJEMPLO DE USO DE UN HTML ESTÁTICO Y UN DINÁMICO 46
FIGURA 2. 14 EJEMPLO DEL FUNCIONAMIENTO DE JAVASCRIPT CON EL MENSAJE “HOLA MUNDO” 47
TABLA 2.1 EVENTOS MÁS COMUNES EN JAVASCRIPT 52
FIGURA 3. 1 ESTRUCTURA DEL PROYECTO (FRONTEND) 55
FIGURA 3. 2 ESTRUCTURA DEL PROYECTO (BACKEND) 55
FIGURA 3. 3 PÁGINA PRINCIPAL DEL CAMPUS MONTERREY CREADA CON HTML, CSS Y JAVASCRIPT 66
FIGURA 3. 4 INTERFAZ DE ADMINISTRADOR PARA EDITAR LA INFORMACIÓN DEL CAMPUS 68

A G R A D E C I M I E N T O S

Con gran humildad y un profundo sentido de gratitud, quiero expresar mi más sincero agradecimiento a quienes han sido pilares fundamentales en este viaje académico.

En primer lugar, agradezco a mis amados padres, por su amor incondicional, su apoyo incansable y su fe inquebrantable en mi capacidad. Este logro es el fruto de su sacrificio y su constante motivación. Sin su presencia y aliento, este camino no habría sido posible.

También agradezco a la Universidad Tecnológica de Puebla (UTP), por ser la institución que me ha brindado la oportunidad de crecer profesionalmente y por su compromiso con la excelencia académica.

Asimismo, extiendo mi más profundo reconocimiento a todos los docentes de la carrera de Tecnologías de la Información por su profesionalismo, dedicación y paciencia han sido invaluables. Gracias por impartir sus conocimientos con pasión y por ser fuente de inspiración, guiando mis pasos y proporcionándome las herramientas necesarias para enfrentar los desafíos de mi futura profesión.

INTRODUCCIÓN

La presente tesina aborda el desarrollo e implementación de un sitio web unificado para Estudios de Posgrado de Excelencia (EP de México), una institución educativa especializada en Educación Médica Continua y Posgrados. Fundada en 2001 en la Ciudad de Puebla, la empresa ha consolidado su presencia a nivel nacional a través de sus cinco campus estratégicamente ubicados en Puebla, Monterrey, Mérida y Guadalajara, además de un campus virtual. Este proyecto surge como una solución tecnológica para centralizar y optimizar la gestión de información de estos campus en una única página web, eliminando la fragmentación y mejorando la experiencia de los usuarios.

El objetivo principal de esta implementación es unificar los cinco campus de EP de México en un solo sitio web, proporcionando una interfaz única y adaptable que pueda mostrar dinámicamente el contenido de cada sede. Esta centralización representa una ventaja competitiva significativa al mejorar la accesibilidad, coherencia de la información y eficiencia en la administración. El sitio web no solo facilita el acceso a la oferta académica, sino que también refuerza el compromiso de la institución con la excelencia académica y su visión de ampliar su alcance educativo en el país.

La metodología de desarrollo se centró en la creación de un sitio web robusto y funcional, dividiendo el trabajo en dos componentes principales: el Frontend y el Backend. Para la interfaz de usuario (Frontend), se emplearon tecnologías web estándar como HTML, CSS y JavaScript, junto con herramientas como Visual Studio Code. Para la lógica del servidor (Backend), se utilizó Visual Studio 2022 Community con ASP.NET Web Form. La interconexión de ambas partes se logra a través de un archivo JSON, lo que permite una gestión de contenido dinámica y eficiente.

Este proyecto se enfoca en el desarrollo de una aplicación web que, a diferencia de un sitio estático, permite la interacción con el usuario y la gestión de datos de manera personalizada. Las limitaciones del proyecto se centran en el alcance de la implementación tecnológica descrita en los capítulos siguientes, sin abordar aspectos de infraestructura de hardware o detalles sobre el número de empleados de la organización, información que no se encuentra disponible en los documentos proporcionados.

Capítulo I

Escuela de Posgrados de Excelencia

EP de México ó Estudios de Posgrado de Excelencia, es una Institución Especializada en Educación Médica Continua y Posgrados con cerca de 180 programas académicos ofertados continuamente. Fundada en la Ciudad de Puebla en el año 2001 nace cimentada sobre la creencia que la educación es el medio para el crecimiento personal y profesional que permite el desarrollo de las sociedades.

Hoy el Sistema EP de México es el único sistema educativo en Educación Media Continua y Posgrados comprometido con la Excelencia académica que brinda atención a profesionales de la salud de 20 estados de la República Mexicana a través de sus 5 campus ubicados estratégicamente:

- Campus Posgrados | Ciudad de Puebla
- Campus Educación Continua | Ciudad de Puebla
- Campus Sur | Ciudad de Mérida
- Campus Occidente | Ciudad de Guadalajara
- Campus Virtual | Ciudad de Puebla

El Campus Virtual cubre las necesidades de alumnos que desean cursar nuestros programas académicos a distancia a través de una plataforma educativa que permite el cumplimiento de los objetivos académicos.

1.1 MISIÓN

Gestionar procesos educativos de excelencia que permitan a nuestros alumnos mejorar sus competencias para ayudar a preservar, mantener y restablecer la salud, seguridad y bienestar de sus pacientes.

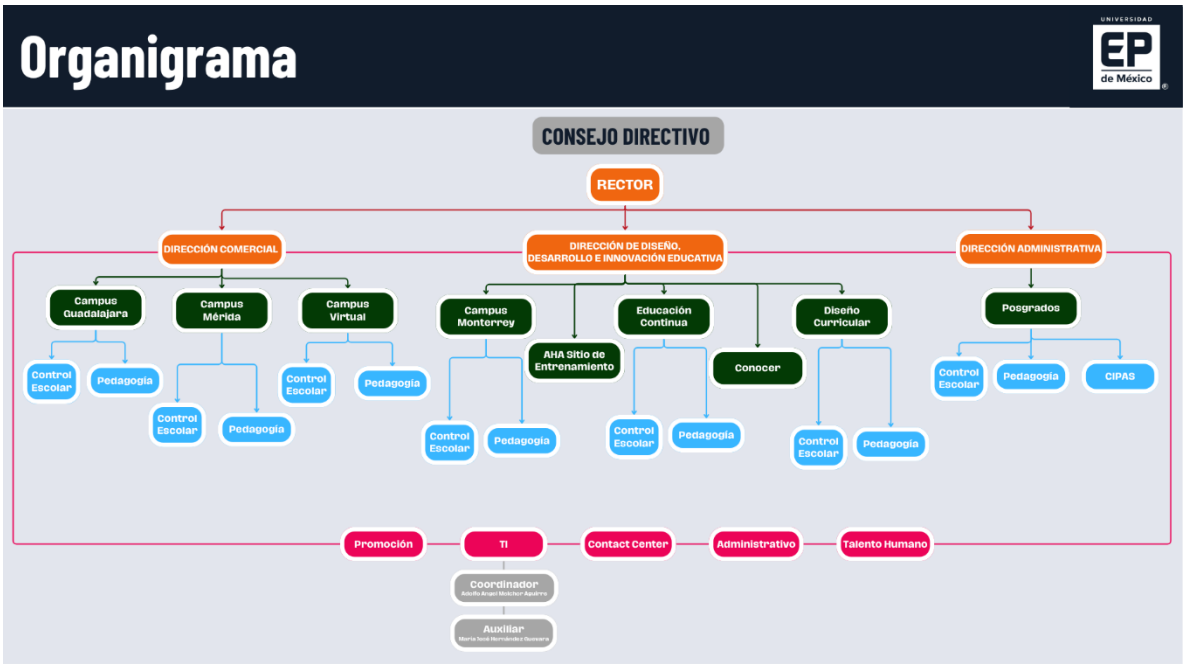
1.2 VISIÓN

Consolidar y ampliar la oferta educativa de educación continua y posgrados enfocada en el área de salud en el país buscando siempre alcanzar la excelencia en cada tarea, a través de la adecuada impartición de programas académicos que brinden las competencias necesarias para el alumnado a través de métodos tradicionales, prácticos y de vanguardia.

1.3 VALORES

Esta institución está formada por el constante trabajo y permanente capacitación que logra la excelencia, cimentada en los siguientes valores fundaméntales: Pasión, Honestidad, Innovación, Sinceridad y Responsabilidad

1.4 ORGANIGRAMA



1.5 PLANTEAMIENTO DEL PROBLEMA

EP de México, institución educativa con presencia en distintos estados del país, actualmente dispone de páginas web independientes para cada uno de sus campus (Monterrey, Mérida, Puebla, Guadalajara y Virtual). Sin embargo, dichas plataformas presentan importantes deficiencias a nivel estructural, estético y funcional, lo que compromete tanto la experiencia del usuario como la imagen institucional como se puede apreciar en la figura 1.1.

Contiene un diseño fragmentado y desactualizado cada campus ha desarrollado su propia versión del sitio web de forma aislada, lo que ha dado lugar a diferencias notables en diseño, navegación y distribución del contenido lo que genera confusión en los visitantes y debilita la identidad visual corporativa. Así mismo su navegación no es unificada ni intuitiva pues los usuarios que desean explorar los distintos campus enfrentan dificultades al no contar con un sistema de navegación centralizado. No existe una estructura común que permita el acceso transversal a la información entre los campus, lo cual reduce la eficiencia en la búsqueda de programas académicos, servicios o datos de contacto.



Figura 1. 1 Diseño y estructura de la página web del campus Puebla

Por otro lado, las páginas actuales no siguen estándares modernos de diseño web, lo que impacta negativamente en la percepción de los visitantes, especialmente entre estudiantes potenciales y padres de familia, quienes esperan una presentación profesional y actualizada de la institución. De igual forma, los sitios web existentes no están completamente optimizados para dispositivos móviles, lo que provoca una experiencia deficiente en smartphones y tablets, afectando a una gran parte del público objetivo que accede desde estos medios.

Estas deficiencias afectan directamente la capacidad institucional para comunicar su oferta educativa, difundir eventos, mantener informada a la comunidad estudiantil y proyectar una imagen acorde a los valores de innovación, accesibilidad y calidad. Además, la multiplicidad de estructuras dificulta la gestión técnica y aumenta los costos de mantenimiento.

Para resolver esta situación, se plantea el diseño y desarrollo de un sitio web unificado que centralice la información de los cinco campus en una sola plataforma, basada en tecnologías web modernas del lado del cliente (HTML, CSS, JavaScript). Este nuevo portal contará con una estructura de código única, flexible y reutilizable, lo que garantizará coherencia visual y funcional en todas las secciones como se puede apreciar en la figura 1.2.

El contenido de cada campus será gestionado dinámicamente mediante un único archivo JSON estructurado, que almacenará datos clave como menús, imágenes, ofertas académicas, ubicación, enlaces a redes sociales y más. Este enfoque permitirá que la visualización del contenido se adapte automáticamente según el campus seleccionado, sin necesidad de múltiples versiones del código.

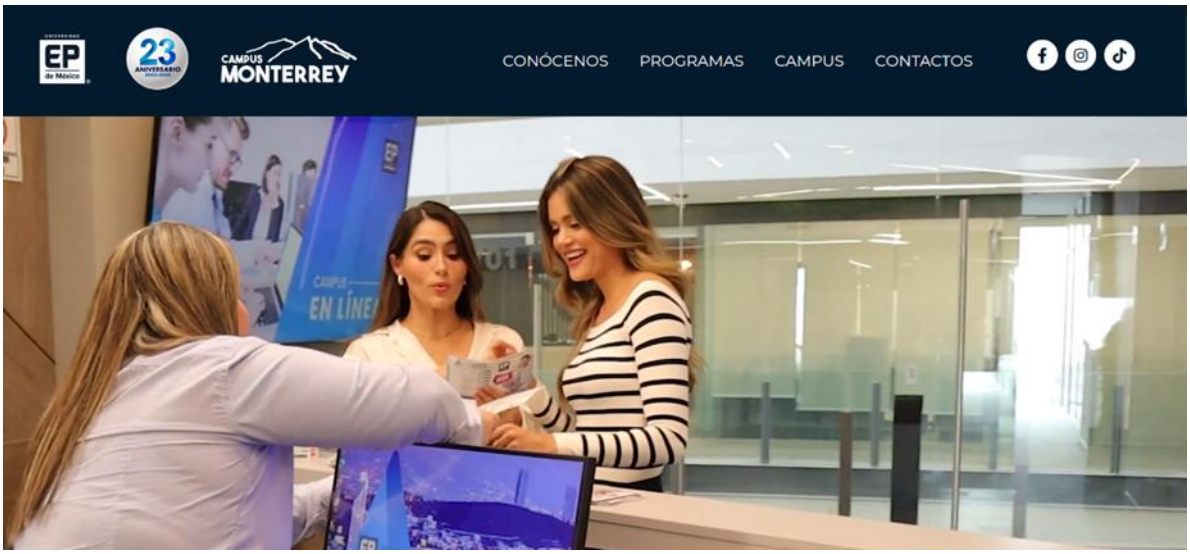


Figura 1. 2 Diseño y estructura del nuevo sitio web para todos los campus

Además, se desarrollará una interfaz de administración (Back-End) sencilla en ASP.NET Web Forms, a través de la cual el personal autorizado podrá editar los contenidos del archivo JSON sin conocimientos técnicos en programación. Esta herramienta permitirá agregar, modificar o eliminar datos desde una interfaz gráfica amigable, optimizando los tiempos de gestión y asegurando la precisión de la información publicada.

Beneficios esperados:

- Imagen institucional moderna, coherente y profesional.
- Mejora en la experiencia del usuario en todos los dispositivos.
- Mayor accesibilidad y navegación intuitiva entre los distintos campus.
- Reducción de tiempos y complejidad en la administración del contenido web.
- Consolidación de una plataforma escalable para futuras actualizaciones.

La creación de un sitio web unificado y administrable representa una necesidad crítica para EP de México en su proceso de modernización digital. Esta solución contribuirá significativamente a mejorar la visibilidad institucional, la satisfacción del usuario y la eficiencia operativa, fortaleciendo así su presencia en el entorno educativo contemporáneo.

1.6 OBJETIVO

Desarrollar una nueva página web unificada para EP de México que integre los cinco campus, estructurada con diseño responsivo y contenido dinámico a través de archivos JSON, utilizando tecnologías web como HTML, CSS y JavaScript. El objetivo es facilitar la navegación, mejorar la experiencia del usuario y permitir al personal administrativo modificar contenidos sin necesidad de conocimientos técnicos.

Capítulo II

Marco Teórico

2.1 FUNDAMENTOS BÁSICOS DE INTERNET

2.1.1 Internet

Internet, la Red o la Autopista de las comunicaciones, es un conjunto de redes descentralizadas de ordenadores configurados para comunicarse entre sí mediante un “lenguaje común” o protocolo. Los orígenes de Internet se remontan al año 1969, en el que la Agencia de Proyectos de Investigación Avanzados de Defensa (DARPA) de los Estados Unidos, estableció la primera conexión entre ordenadores que se denominó ARPANET como se puede apreciar en la figura 2.1. El objetivo de esta red primigenia era descentralizar la información militar estratégica de forma que cada componente de la red pudiera acceder a dicha información de manera independiente al resto de las máquinas. Así pues, si cualquiera de los nodos de la red era destruido o dejaba de funcionar, el resto de los componentes seguirían siendo totalmente funcionales. La primera conexión de ordenadores se estableció entre las Universidades de California e Utah.

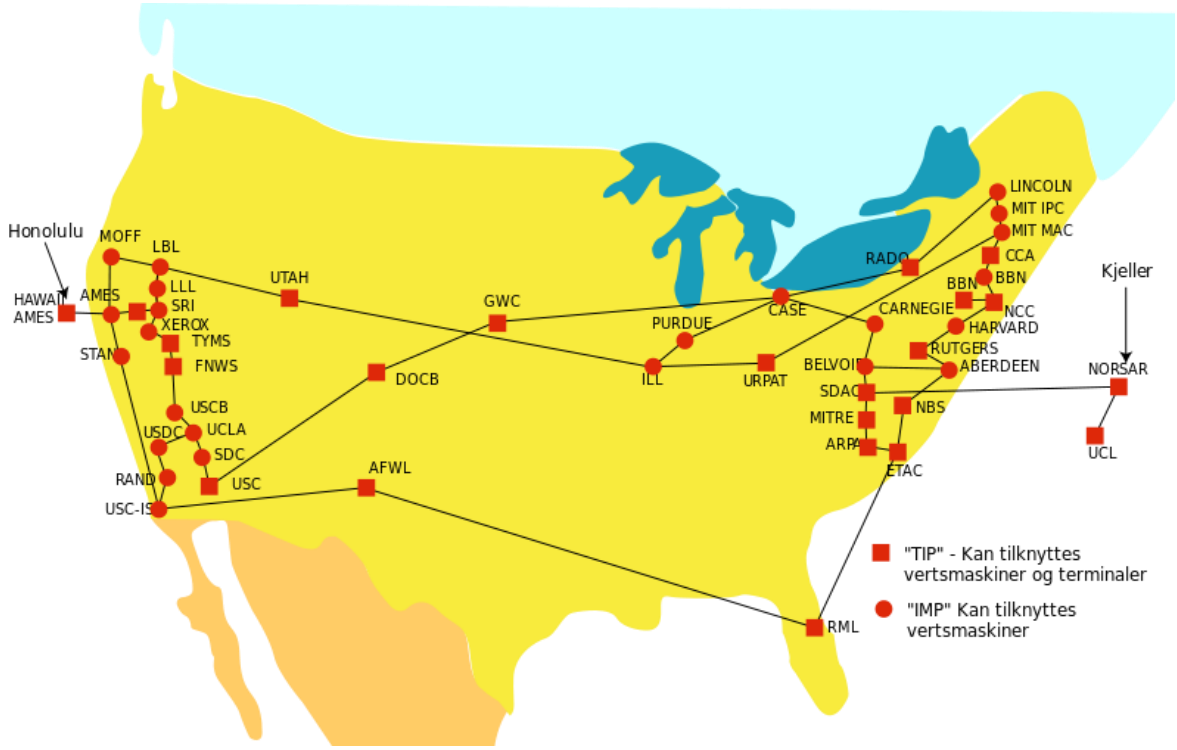


Figura 2. 1 Primera conexión entre ordenadores que se denominó ARPANET

Las conexiones entre ordenadores se establecen por medio de una serie de protocolos TCP/IP, unos "lenguajes comunes" que hablan todos los ordenadores conectados a Internet. El más popular de todos ellos, es el protocolo http, que se utiliza para acceder a las páginas Web.

2.1.2 La Web

Uno de los servicios más utilizados de Internet sea la Web o World Wide Web (WWW o W3) que está formada por un conjunto de protocolos mediante los cuales es posible compartir información entre ordenadores mediante un formato de archivo que se conoce como hipertexto, este formato de archivo permite recorrer fácilmente un conjunto de documentos enlazados a través de una serie de enlaces, hipervínculos o referencias cruzadas. La Web fue creada por Tim Berners-Lee como se puede apreciar en la figura 2.2, que está basada en el desarrollo de Internet y empezó a formarse en Suiza a principios de 1990 en la Organización Europea para la Investigación Nuclear (CERN). El protocolo sobre el que se fundamente la Web es http (Protocolo de Transferencia de Hipertexto), que especifica la forma en la que se comunican el ordenador donde se almacena un archivo Web (servidor) y el ordenador en el que se desea consultar dicho archivo (cliente).

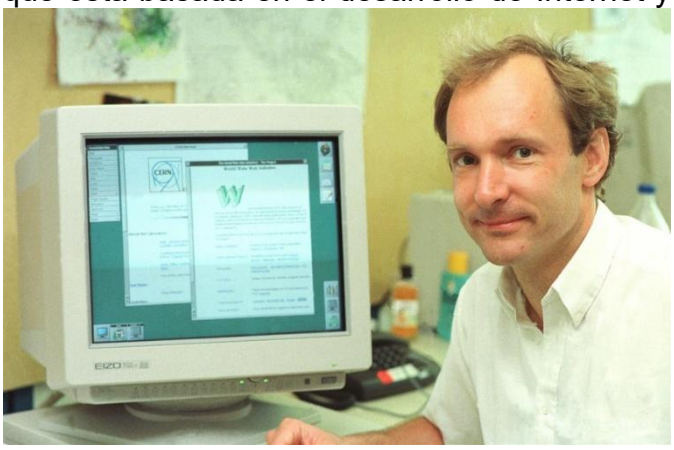


Figura 2. 2 Tim Berners-Lee creador de la Web

Tim O'Reilly acunó en el año 2004 el término Web 2,0 que define una segunda generación de la Web en la que predominan los servicios basados en comunidades de usuarios como las redes sociales, los blogs o los wikis, cuyo objetivo es promover el intercambio de información entre los usuarios de Internet. En 2006 se habla por primera vez de Web 3.0, en un artículo elaborado por Jeffrey Zeldman asociando esta terminología a nuevas tecnologías tales como AJAX.

2.1.3 Páginas y Sitios Web

Una página web es un documento electrónico o digital que forma parte de un sitio web y al que se puede acceder a través de un navegador web. Está escrita en lenguajes de marcado como HTML (HyperText Markup Language) puede incluir funciones especiales desarrolladas en otros lenguajes de programación tales como Javascript, PHP, ASP, etc. Así mismo puede contener texto, imágenes, videos, audio y enlaces a otras páginas o recursos. Una página web es, en esencia, un archivo estático o dinámico que se visualiza en un navegador. que también

Un sitio Web es un conjunto de páginas Web relacionadas entre sí y agrupadas alrededor de un dominio de Internet (un fragmento de la Red identificado con diversos dispositivos o equipos conectados a Internet). La navegación a través de las distintas páginas de un sitio Web se realiza habitualmente por medio de enlaces o hipervínculos, aunque una página Web también puede contener vínculos a otros sitios Web y dominios diferentes.

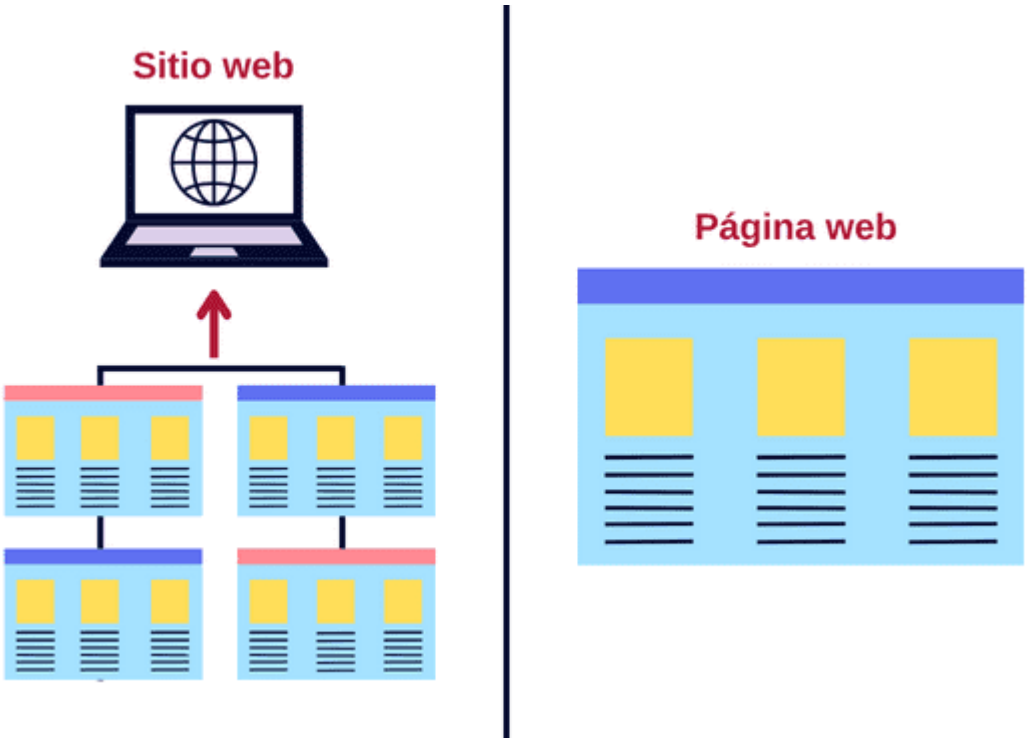


Figura 2. 3 Diferencia entre Sitio Web y Página Web

En una clasificación generalista, las páginas Web se pueden dividir en estáticas (de contenido predeterminado o dinámicas (con un contenido cambiante generado en el momento de ser solicitadas). Generalmente, cualquier página Web está compuesta tanto por texto como por otros elementos multimedia tales como imágenes, sonido y vídeos o animaciones, podemos apreciar la figura 2.3 para entender la diferencia entre página Web y sitio Web.

También decimos que una aplicación web es un software que se ejecuta en un servidor web y al que los usuarios acceden a través de un navegador web. A diferencia de una página o sitio web estático que principalmente muestra información, una aplicación web permite la interacción del usuario y realiza funciones específicas, como procesar datos, gestionar bases de datos, o proporcionar funcionalidades dinámicas y personalizadas. Ejemplos comunes incluyen servicios de correo electrónico en línea, plataformas de redes sociales, sistemas de gestión de contenido (CMS) y aplicaciones de comercio electrónico.

2.1.4 Tipos de sitios Web

Una agrupación de documentos o páginas Web que se almacenan en un mismo servidor se conoce con el nombre de "sitio Web". Algunos de los principales tipos o categorías de sitios Web que se pueden encontrar en la Red son:

Escaparates comerciales: Son sitios donde se muestran los servicios o productos que ofrece una determinada empresa o profesional. Habitualmente, forman parte de la imagen corporativa de la propia empresa, son un componente más de una campaña publicitaria o de promoción de marca a nivel global o, incluso, disponen

una plataforma de comercio electrónico desde la que se desarrolla la actividad económica de dicha empresa. Para que uno de estos sitios Web tenga éxito, deben incluir información que ayude a los visitantes a conocer la empresa y sus productos.

- **Portales o sitios Web de contenidos:** Muestran información sobre un tema específico o sobre diferentes temas relacionados o no entre sí. Pueden ser portales horizontales con todo tipo de informaciones como el de Yahoo! (http://www.yahoo.es) o verticales, portales desarrollados alrededor de un tipo de información específico como el del Consejo Superior de Deportes (http://www.csd.gob.es). Un tipo particular de portal de contenidos son los blogs.
- **Comercio electrónico:** Sitios exclusivamente dedicados a la compraventa de servicios o productos como por ejemplo Amazon (http://www.amazon.com) o eBay (http://www.ebayanuncios.es/ebay).
- **Comunidades:** Sitios donde los visitantes participan de una manera activa creando sus propios contenidos. Ejemplos de este tipo de comunidades son los foros o las redes sociales como lo son YouTube (http://www.youtube.com) o Facebook (http://www.facebook.com).
- **Buscadores:** Sitios específicamente diseñados para la búsqueda de información en Internet (también conocidos como motores de búsqueda). Algunos de los buscadores más conocidos son Google (http://www.google.es) o bing (http://www.bing.com), aunque muchos portales como Yahoo! también incluyen sus propios motores de búsqueda.

2.1.5 Navegadores y Herramientas para Desarrolladores

2.1.5.1 Microsoft Edge

Microsoft Edge es un navegador web desarrollado por la empresa Microsoft. Fue lanzado en 2015 para reemplazar a Internet Explorer. Desde entonces, ha evolucionado y mejorado para ofrecer una experiencia de navegación más rápida, segura y eficiente. Y lo mejor es que se encuentra disponible para los sistemas operativos Windows, Mac y Linux.

Sus principales características son:

- Una herramienta para organizar contenido web en grupos, facilitando la gestión de ideas y la recuperación de información importante.
- Permite a los usuarios mover las pestañas al lado izquierdo de la pantalla para una mejor visualización y gestión, además de agruparlas y darles nombre.
- Microsoft Edge incluye características como SmartScreen para proteger contra phishing y malware, y Monitor de contraseñas para ayudar a mantener la seguridad de las contraseñas.
- Un asistente de IA integrado que ayuda con tareas como responder preguntas complejas, resumir contenido, generar borradores y crear imágenes, todo directamente desde el navegador.

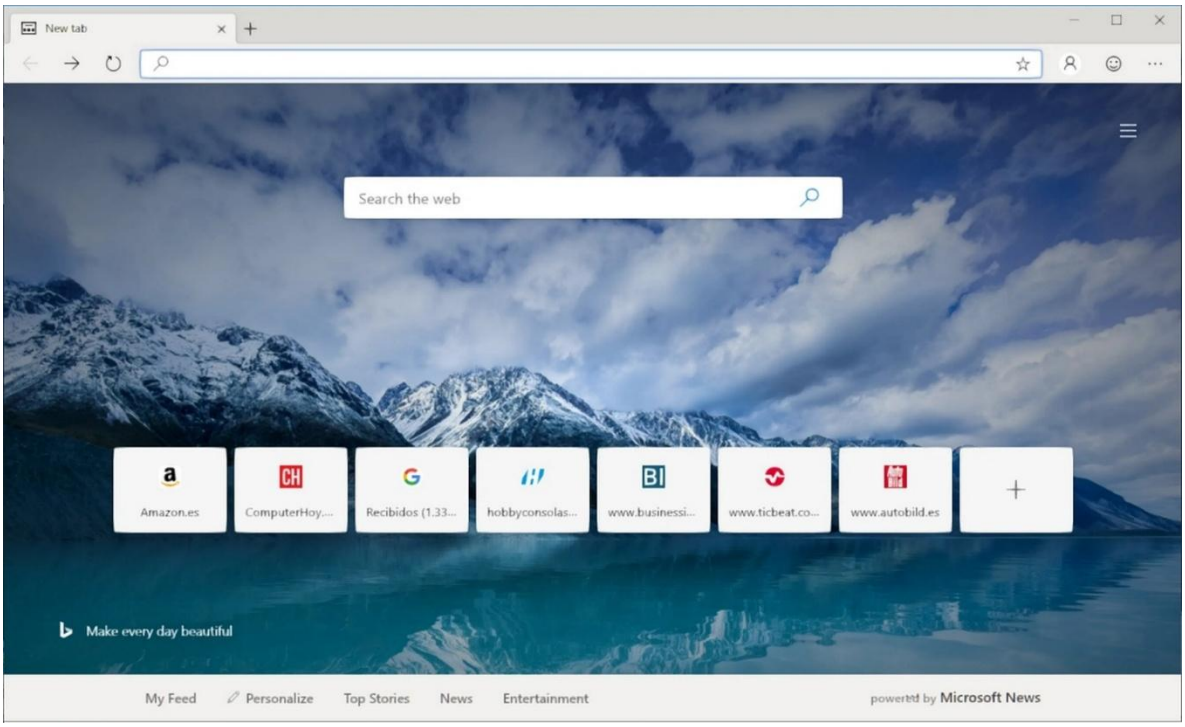


Figura 2. 4 Navegador Internet Explorer

2.1.5.2 Google Chrome

Es un navegador web desarrollado por Google véase la figura 2.6. Es conocido por su velocidad, seguridad y simplicidad, y es uno de los navegadores más utilizados a nivel mundial. Chrome permite a los usuarios acceder a sitios web, realizar búsquedas y utilizar aplicaciones web. También ofrece funciones como sincronización de datos entre dispositivos, extensiones y un administrador de tareas integrado, actualmente está disponible para cualquier dispositivo.

Entre sus características están:

- Diseño centrado en la velocidad, con un inicio más rápido y una carga instantánea de páginas Web. También permite realizar búsquedas en la Web a gran velocidad.
- Tiene pocos botones, barras o menús. De hecho, la barra de herramientas se encuentra en lo más alto de la pantalla y así deja a la vista casi todo el contenido de la página que se visita.
- Google Chrome incorpora la función de traducción automática dentro del propio navegador sin necesidad de complementos o extensiones adicionales. Cuando el idioma de una página Web no coincide con el idioma definido en las preferencias del navegador, Chrome solicita de forma automática la traducción del documento al idioma seleccionado.
- Cuenta con un sistema de pestañas que permiten tener varias páginas disponibles en una sola ventana.

- Funciones de seguridad que protegen el equipo contra sitios Web malintencionados gradas a tecnologías como la navegación segura, zonas de pruebas y actualizaciones automáticas. Google Chrome también incluye configuraciones de seguridad para la detección de phishing y advertencias sobre la posibilidad de software malintencionado.

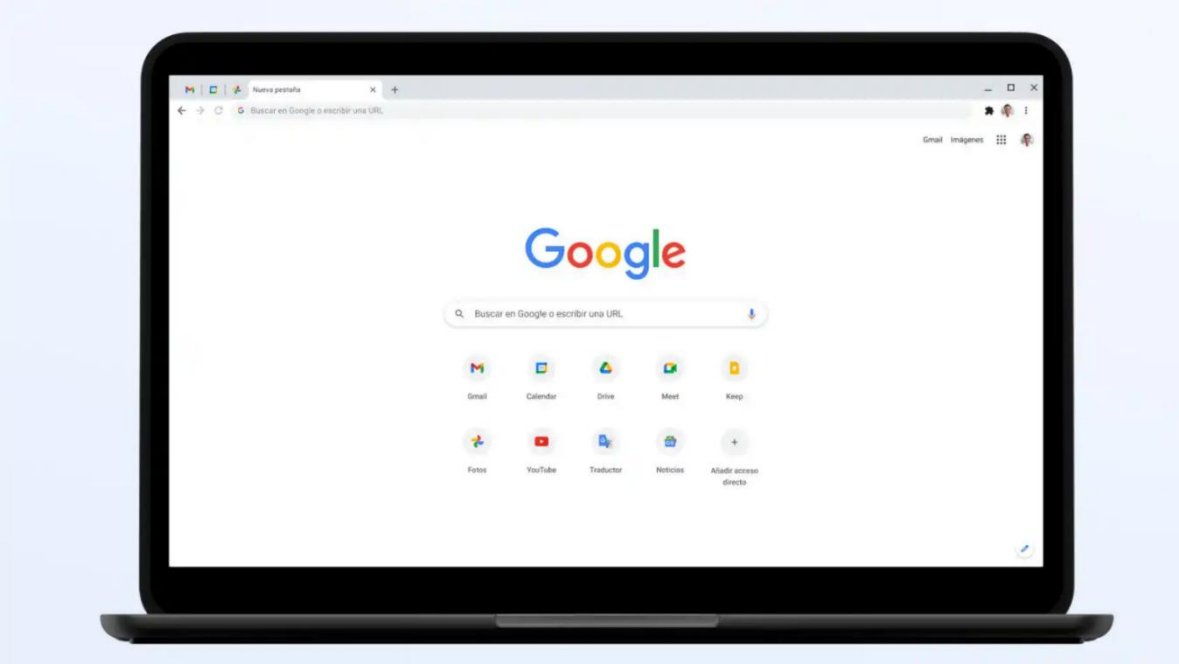


Figura 2. 5 Navegador Google Chrome

2.1.5.3 Firefox

Mozilla Firefox es un navegador de carácter libre y gratuito basado en el motor de renderización de Gecko, donde se implementan los estándares Web más actualizados para la correcta interpretación de páginas como se muestra en la figura 2.7.

Algunas de sus características principales son:

- Mejoras en la seguridad y privacidad del navegador que permiten establecer la legitimidad de un sitio Web con el que se desea operar o eliminar todo rastro de la visita a un determinado sitio Web sobre el que se tenga cualquier tipo de dudas.
- Mejoras en el rendimiento a través de una mejor protección ante fallos o una gestión mejor de la memoria, Gracias al nuevo motor de presentación y trazado del programa, las páginas Web se cargarán más rápido.
- Más de 6.000 formas de personalizar el programa con la utilización de una numerosa biblioteca de complementos. El gestor de complementos de Firefox permite activar o desactivar fácilmente los distintos complementos instalados.

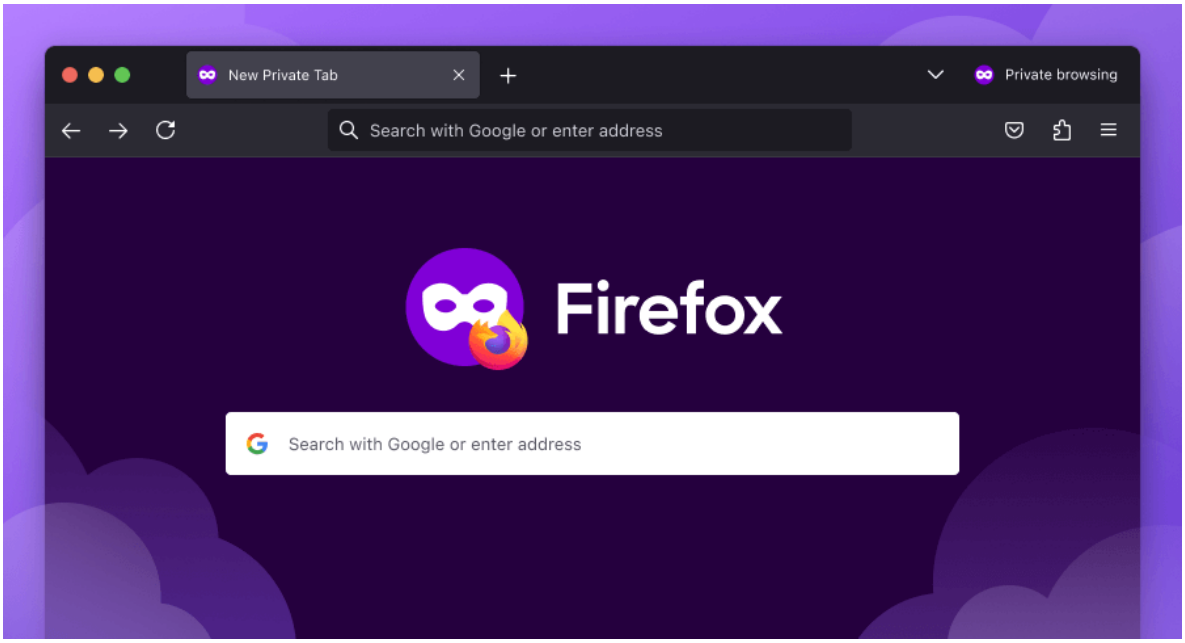


Figura 2. 6 Navegador Firefox

2.1.6 Otros Servicios de Internet

2.1.6.1 Correo Electrónico

El correo electrónico es uno de los servicios de Internet más importantes, su función es la de permitir intercambiar mensajes entre usuarios de la Red de una manera rápida y sencilla vea la figura 2.8. Un mensaje de correo electrónico o e-mail es un mensaje que se transmite entre dos usuarios a través de Internet, generalmente mediante un protocolo SMTP, estos mensajes pueden transmitir tanto texto como todo tipo de documentos digitales. En el año 1961, fue presentado en el Instituto Tecnológico de Massachusetts (MIT) una primitiva herramienta similar a la del correo electrónico que permitía compartir información entre ordenadores y almacenarla en archivos en el disco de cada equipo. En el año 1965 empezó el correo electrónico propiamente dicho y en el año 1966 ya se había extendido en distintas redes de ordenadores, para poder enviar y recibir un mensaje de correo electrónico, tanto el remitente como el destinatario deben disponer de una dirección de correo electrónico, proporcionada por un proveedor de correo. Dicho proveedor habitualmente suele ser el mismo proveedor de acceso a Internet (la compañía con la que el usuario contrata su acceso a la Red) o bien cualquiera de los servicios gratuitos de correo electrónico disponibles en Internet. Una dirección de correo electrónico identifica de forma única a cada uno de los



Figura 2. 7 Correo Electrónico como un servicio de Internet

usuarios del servicio y su estructura genérica es la siguiente: **nombre_usuario@proveedor.dominio,** observe la figura 2.9, donde **nombre_usuario** es el nombre o código que se asigna de forma única a un determinado usuario del proveedor de correo electrónico, **proveedor** es el nombre del proveedor del servicio y **dominio** es el nombre de identificación de la red a la que pertenece el proveedor Para crear y recibir mensajes de correo electrónico se puede utilizar un programa específico denominado "cliente de correo" en el ordenador o una interfaz Web a la que se accede a través de un navegador.



Figura 2. 8 Estructura genérica de un Correo Electrónico

2.1.6.2 FTP

FTP son las siglas en inglés de File Transfer Protocol (protocolo de transferencia de archivos), un protocolo para la transferencia de archivos entre ordenadores conectados a una red TCP. Gracias a este servicio, desde un ordenador determinado, llamado cliente, se puede conectar con otro equipo debidamente acondicionado llamado ordenador servidor y descargar de él de manera directa archivos o bien enviarle archivos, independientemente de los sistemas operativos instalados en cada uno de estos equipos. El servicio FTP está pensado para ofrecer una elevada velocidad en la transferencia de archivos, aunque adolece de una seguridad deficiente, ya que el único requisito de seguridad necesario para establecer una conexión FTP es la especificación de un nombre de usuario y una contraseña, sin que exista ningún tipo de cifrado.

En el año 1971 el Instituto Tecnológico de Massachusetts (MIT) planteó las primeras bases del protocolo FTP, que quedaría definitivamente establecido en 1985, en la forma en que se conoce aún hoy en día. La comunicación entre el ordenador cliente y servidor es independiente del sistema de archivos que se utilice en cada uno de los equipos (y por tanto, no importan que los sistemas operativos sean distintos en cada caso) y, además, la conexión que se establece es de carácter bidireccional, es decir, se pueden enviar y recibir datos en cada una de las partes de manera simultánea.

El servidor FTP es un ordenador donde se instala un programa especial que permite el intercambio de datos entre diferentes ordenadores o servidores. Como cliente FTP se puede utilizar cualquier navegador Web con capacidad para el intercambio de datos o recurrir a programas específicos disponibles en el mercado.

2.1.6.3 P2P

P2P es la abreviatura que se utiliza para identificar las redes llamadas de pares, entre iguales o punto a punto (peer to peer) observe la figura 2.10. A diferencia de otras redes y servicios estudiados hasta el momento, una red P2P funciona de manera que ninguno de los ordenadores que la componen es cliente ni servidor, sino que todos ellos se comportan como iguales (o lo que es lo mismo, actúan de manera simultánea como clientes y servidores frente a los restantes equipos de la red). El objetivo de una red P2P es permitir el intercambio directo de información entre dos ordenadores cualesquiera. Además, este tipo de redes se caracteriza por la optimización del ancho de banda de las conexiones, consiguiendo de esta manera un mejor rendimiento en las transferencias de archivos que con otros mecanismos centralizados tradicionales donde el ancho de banda depende principalmente de la capacidad disponible en el servidor. Habitualmente las redes P2P se utilizan para compartir archivos (audio, vídeo, software) entre usuarios, aunque también tiene su aplicación en telefonía VolP y sirven para mejorar la eficiencia de la transmisión de datos en tiempo real. Algunas de las principales redes P2P más conocidas en Internet son Ares, BitTorrent, eDonkey, GNUnet, Gnutella, Kad, Napster, P2PTV, Usenet o WPNP.

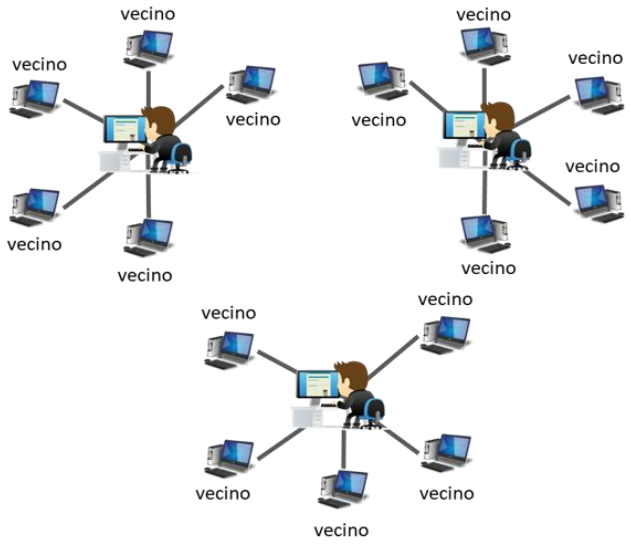


Figura 2. 9 P2P Redes llamadas de pares, entre iguales o punto a punto

2.1.7 Redes Sociales

De forma genérica, una red social es cualquier estructura social compuesta por un grupo de individuos conectados mediante diferentes tipos de relaciones como relaciones de amistad o parentesco en algún grado o bien que se mueven por intereses comunes o comparten algún tipo de conocimiento en común.

En Internet, las redes sociales han suscitado discusiones como la de falta de privacidad, pero también han servido como medio de convocatoria para manifestaciones públicas en protestas. Estas plataformas crearon, también, una nueva forma de relación entre empresas y clientes, abriendo caminos tanto para la

interacción, como para el anuncio de productos o servicios, puede observar la figura 2.10.

Algunas de las redes sociales con más difusión hoy en día en Internet son las siguientes:

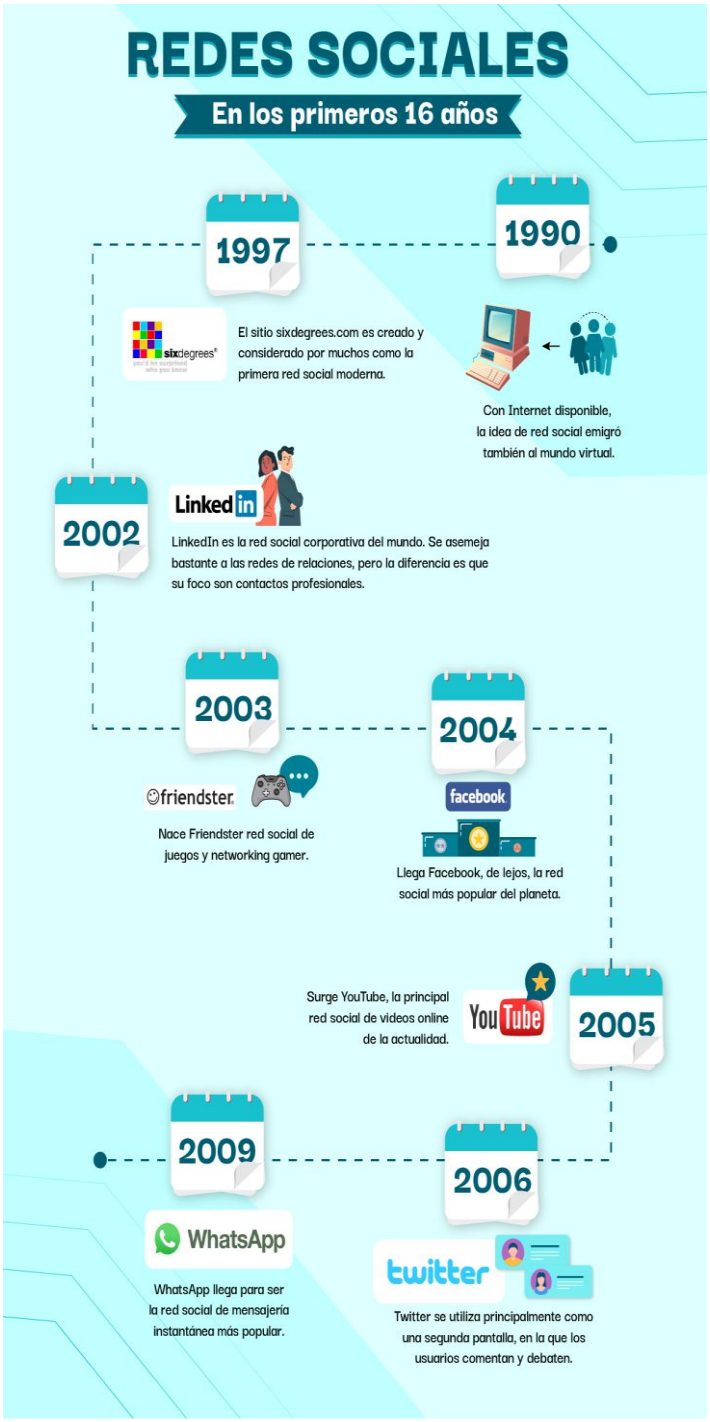


Figura 2. 10 Historia de las Redes Sociales

- **Instagram.** Si bien se puede pensar que esta plataforma es un banco de grandes fotografías personales y *"reels"*, la realidad es que su consumo nos ha despertado cercanía, entretenimiento y actividades para hacer con amigos y familiares.
- **LinkedIn.** La red social por excelencia de creación de relaciones profesionales, networking y solución de necesidades laborales: desde un nuevo integrante al equipo de trabajo hasta el próximo invitado al evento de la empresa.
- **Twitter.** Que para su caso es la muestra de comportamientos sociales, libertad de expresión y luchas lideradas por múltiples comunidades.
- **Facebook.** Que en el 2004 fue creada junto con un par de amigos, en una habitación universitaria, para ofrecer una membresía a los estudiantes de Harvard, luego a más universidades y así ingresar al sitio web para conectar con una red de amigos comunes.
- **Tinder.** Si bien es una aplicación de citas, su uso completo por personas interesadas en conversar con otros y conseguir un momento de interacción, la convierten en la red social más importante para citas.

2.2 HERRAMIENTAS DE PROGRAMACIÓN

2.2.1 Lenguajes

2.2.1.1 HTML5

HTML5 (HyperText Markup Language) es el estándar actual para estructurar y presentar contenido en la web. Es un lenguaje basado en un conjunto de códigos (etiquetas) que le indican al navegador web cómo interpretar la información de una página. Un documento HTML5 es básicamente un archivo de texto plano, lo que lo hace compatible con cualquier navegador y sistema operativo.

Las etiquetas HTML5 representan distintos elementos de una página web, como <title>, <head>, <body>, <p>, entre otras. Estas etiquetas pueden incluir parámetros o atributos opcionales que modifican su comportamiento, por ejemplo: <img src="imagen.jpg" alt="Descripción de la imagen">.

Ventajas:

- **Soporte Multimedia Nativo:** Permite incrustar audio y video directamente con las etiquetas <audio> y <video>, eliminando la necesidad de plugins de terceros como Flash.
- **Mejor Semántica:** Introduce etiquetas como <header>, <footer>, <nav>, <article>, y <section>, que mejoran la estructura del documento, haciendo el código más legible y optimizado para motores de búsqueda y tecnologías de asistencia.
- **Compatibilidad Multiplataforma:** Diseñado para funcionar en una amplia gama de dispositivos (escritorio, tabletas, smartphones) y navegadores, facilitando el diseño web adaptable (responsive design).
- **Funcionalidades Offline:** Con Application Cache, las aplicaciones web pueden almacenar recursos y funcionar incluso sin conexión a internet.
- APIs Avanzadas: Incluye nuevas APIs para geolocalización, arrastrar y soltar (drag and drop), almacenamiento local (localStorage), y gráficos interactivos con <canvas> y WebGL.
- **Formularios Mejorados:** Ofrece nuevos tipos de entrada para formularios (ej., email, date, number) y validación nativa, mejorando la experiencia del usuario.

Desventajas:

- **Soporte de Navegadores Antiguos:** Algunas características de HTML5 pueden no ser totalmente compatibles con navegadores muy antiguos, aunque esto es cada vez menos común.
- **Curva de Aprendizaje:** Aunque es accesible, dominar todas las nuevas etiquetas, APIs y las mejores prácticas junto con CSS3 y JavaScript puede requerir tiempo.

- **Fragmentación de Codecs Multimedia:** Si bien soporta audio y video, los diferentes navegadores pueden no soportar los mismos formatos de codec, lo que a veces obliga a proporcionar múltiples versiones de un archivo multimedia.
- **Dependencia de JavaScript:** Para una interactividad avanzada y lógica de aplicación compleja, HTML5 sigue dependiendo en gran medida de JavaScript.

2.2.1.2 JavaScript

JavaScript es un lenguaje de programación de alto nivel, interpretado y orientado a objetos, fundamental para crear **interactividad y dinamismo** en las páginas web. Mientras HTML define la estructura y el contenido, JavaScript permite agregar comportamientos como animaciones, validación de formularios, manipulación del contenido de la página (DOM) y comunicación asíncrona con servidores.

Es predominantemente un **lenguaje del lado del cliente**, lo que significa que su código se ejecuta directamente en el navegador del usuario, haciendo que las interacciones sean rápidas y eficientes. Sin embargo, con la llegada de **Node.js**, JavaScript también puede ejecutarse en el lado del servidor, permitiendo el desarrollo de aplicaciones web completas (full-stack) con un solo lenguaje.

Ventajas:

- **Interactividad Dinámica:** Permite crear experiencias de usuario ricas y dinámicas sin necesidad de recargar la página completa.
- **Soporte de Navegadores:** Es universalmente soportado por todos los navegadores web modernos.
- **Versatilidad (Full-Stack):** Gracias a Node.js, puede usarse tanto en el frontend como en el backend, lo que facilita el desarrollo y el mantenimiento.
- **Gran Ecosistema:** Cuenta con una vasta cantidad de frameworks y librerías (como React, Angular, Vue.js para frontend y Express.js para backend) que aceleran el desarrollo.
- **Comunidad Amplia:** Posee una de las comunidades de desarrolladores más grandes, lo que se traduce en mucha documentación, tutoriales y soporte.

Desventajas:

- **Problemas de Seguridad (Frontend):** Al ejecutarse en el cliente, el código JavaScript es visible para el usuario, lo que puede exponer vulnerabilidades si no se maneja correctamente.
- **Dependencia del Navegador:** El rendimiento y la compatibilidad pueden variar ligeramente entre diferentes navegadores.
- **Dificultad con Tareas Complejas de CPU:** No es ideal para tareas que requieren un procesamiento intensivo de la CPU en el lado del cliente.

- **Posibilidad de Desactivación:** Aunque raro, los usuarios pueden deshabilitar JavaScript en sus navegadores, lo que podría afectar la funcionalidad de una web.

2.2.1.3 Ajax

AJAX (Asynchronous JavaScript and XML) no es un lenguaje de programación, sino una **técnica de desarrollo web** que permite a las aplicaciones web comunicarse con un servidor de forma **asíncrona** en segundo plano, sin necesidad de recargar la página completa. Antes de AJAX, cada interacción con el servidor (como enviar un formulario o cargar datos) implicaba una recarga total de la página, lo que interrumpía la experiencia del usuario.

AJAX utiliza objetos **XMLHttpRequest** (o la más moderna API fetch) en JavaScript para enviar y recibir datos del servidor. Los datos suelen intercambiarse en formatos como **XML** o, más comúnmente hoy en día, **JSON** (JavaScript Object Notation). Esta técnica mejora significativamente la interactividad y la velocidad de las aplicaciones web al actualizar solo las partes necesarias de la página.

Ventajas:

- **Experiencia de Usuario Mejorada:** Permite actualizaciones de contenido sin recargar la página, lo que resulta en una navegación más fluida y rápida.
- **Menor Uso de Ancho de Banda:** Solo se transfieren los datos necesarios, no toda la página, lo que reduce el consumo de ancho de banda y mejora el rendimiento.
- **Asincronía:** Las solicitudes al servidor se realizan en segundo plano, permitiendo que el usuario siga interactuando con la página mientras se esperan los datos.
- **Interactividad en Tiempo Real:** Facilita la creación de aplicaciones con actualizaciones en tiempo real, como chats o notificaciones.

Desventajas:

- **SEO:** El contenido cargado dinámicamente con AJAX puede ser más difícil de indexar para los motores de búsqueda si no se implementa correctamente.
- **Historial del Navegador:** Los cambios de estado de la página no se reflejan automáticamente en el historial del navegador, lo que puede dificultar el uso de los botones "atrás" y "adelante".
- **Seguridad:** Las solicitudes asíncronas deben manejarse con cuidado para evitar vulnerabilidades como la inyección de código.
- **Complejidad en la Depuración:** Depurar aplicaciones con muchas solicitudes AJAX puede ser más complejo debido a la naturaleza asíncrona.

2.2.1.4 PHP

PHP (Hypertext Preprocessor) es un lenguaje de programación de script de **código abierto** diseñado específicamente para el **desarrollo web del lado del servidor**. El código PHP se ejecuta en el servidor web antes de que la página sea enviada al navegador del usuario. El servidor procesa el código, genera HTML (o cualquier otro formato) y envía el resultado final al cliente.

PHP es ampliamente utilizado para crear sitios web dinámicos, aplicaciones web complejas, sistemas de gestión de contenido (CMS) como **WordPress**, **Joomla!** y **Drupal**, y se integra fácilmente con bases de datos como MySQL, PostgreSQL y Oracle. Su sintaxis es similar a C y Perl, lo que lo hace relativamente fácil de aprender.

Ventajas:

- **Fácil de Aprender y Usar:** Su sintaxis es intuitiva, lo que lo hace accesible para principiantes.
- **Amplio Soporte de Bases de Datos:** Compatible con la mayoría de las bases de datos populares.
- **Multiplataforma:** Se puede ejecutar en diversos sistemas operativos como Linux, Windows y macOS.
- **Gran Comunidad y Ecosistema:** Dispone de una enorme comunidad, lo que significa mucha documentación, frameworks (Laravel, Symfony, CodeIgniter) y librerías disponibles.
- **Rendimiento:** Las versiones modernas de PHP (PHP 7 y 8) han mejorado significativamente su rendimiento.
- **Código Abierto y Gratuito:** Es de uso libre y su código fuente está disponible.

Desventajas:

- **Ejecución del Lado del Servidor:** Cada solicitud requiere que el servidor procese el script PHP, lo que puede ser más lento que el JavaScript del lado del cliente para ciertas interacciones.
- **Saturación del Servidor:** Para sitios de muy alto tráfico, una mala optimización del código PHP puede llevar a una alta carga del servidor.
- **Inconsistencias en el Diseño:** Históricamente, PHP tenía algunas inconsistencias en el diseño de sus funciones, aunque esto se ha mejorado en versiones recientes.
- **Curva de Rendimiento:** Aunque ha mejorado, en ciertos escenarios intensivos en computación, otros lenguajes de backend podrían ser más rápidos.

2.2.1.5 ASP

ASP (Active Server Pages) es una tecnología de Microsoft para el **desarrollo de páginas web dinámicas del lado del servidor**. Similar a PHP, ASP procesa scripts en el servidor web (generalmente **Internet Information Services o IIS** de Microsoft) y genera HTML que luego se envía al navegador del cliente.

Originalmente, ASP utilizaba lenguajes de scripting como VBScript o JScript. Su sucesor moderno es **ASP.NET**, un marco de desarrollo web robusto que forma parte de la plataforma .NET de Microsoft. ASP.NET permite construir aplicaciones web empresariales, servicios web y APIs utilizando lenguajes como **C#** o **Visual Basic .NET**, ofreciendo herramientas avanzadas, mayor rendimiento y seguridad en comparación con el ASP clásico.

Ventajas:

- **Integración con el Ecosistema Microsoft:** Se integra perfectamente con otras tecnologías y herramientas de Microsoft (Visual Studio, SQL Server, Azure).
- **Potencia y Escalabilidad:** ASP.NET es un framework muy potente y escalable, adecuado para aplicaciones empresariales grandes y complejas.
- **Alto Rendimiento:** Ofrece un excelente rendimiento debido a la compilación del código (a diferencia de los lenguajes interpretados puros).
- **Seguridad:** Incluye características de seguridad robustas y es activamente mantenido por Microsoft.
- **Amplia Librería (Framework .NET):** Se beneficia de la vasta y rica biblioteca de clases de .NET, que proporciona soluciones para casi cualquier tarea.

Desventajas:

- **Plataforma Propietaria:** Es una tecnología de Microsoft, lo que puede implicar costos de licenciamiento (para algunos componentes o herramientas específicas) y menor flexibilidad para hospedar en servidores que no sean Windows.
- **Curva de Aprendizaje:** Puede tener una curva de aprendizaje más pronunciada que PHP, especialmente para desarrolladores nuevos en el ecosistema .NET.
- **Menor Comunidad Fuera de Microsoft:** Aunque su comunidad es grande, es más centralizada en el ámbito de Microsoft en comparación con lenguajes como JavaScript o PHP.
- **Peso del Framework:** El framework .NET puede ser percibido como más pesado y complejo para proyectos pequeños o sencillos.

2.2.1.6 JSON

JSON (JavaScript Object Notation) es un formato de texto ligero para el intercambio de datos. A pesar de su nombre, que viene de JavaScript, JSON es un formato completamente independiente del lenguaje. Esto significa que puede ser utilizado y comprendido por casi cualquier lenguaje de programación, como Python, Java, PHP, Ruby, C#, entre otros.

Se usa principalmente para transmitir datos entre un servidor y una aplicación web (o móvil), y a menudo se prefiere sobre XML por ser más simple y ligero. JSON es fácil de leer y escribir para los humanos, y fácil de analizar y generar para las máquinas.

Ventajas

- **Legibilidad Humana:** Su sintaxis es muy intuitiva y fácil de leer y escribir para las personas, lo que facilita la depuración y el mantenimiento del código.
- **Ligereza y Eficiencia:** Utiliza una sintaxis concisa, lo que resulta en archivos más pequeños en comparación con otros formatos como XML. Esto se traduce en transferencias de datos más rápidas y un menor consumo de ancho de banda.
- **Facilidad de Análisis (Parsing):** La mayoría de los lenguajes de programación modernos tienen funciones o librerías nativas o de fácil acceso que permiten analizar (parsear) y generar datos JSON con muy poco código. Esto simplifica la integración y el manejo de datos entre diferentes sistemas.
- **Independencia del Lenguaje:** Aunque su nombre deriva de JavaScript, JSON es un formato universal. Puede ser utilizado y entendido por casi cualquier lenguaje de programación, lo que lo hace ideal para la comunicación entre sistemas desarrollados en diferentes tecnologías.
- **Versatilidad:** Es ampliamente utilizado en el desarrollo web (APIs RESTful), aplicaciones móviles, bases de datos NoSQL (como MongoDB), configuraciones de aplicaciones y en muchos otros escenarios donde se requiere el intercambio de datos estructurados.

Desventajas

- **Falta de Esquema o Validación Formal:** A diferencia de XML, JSON por sí mismo no tiene un mecanismo integrado para definir un esquema (como XSD en XML). Esto significa que la validación de la estructura y los tipos de datos debe manejarse en el código de la aplicación, lo que puede ser propenso a errores si no se implementa cuidadosamente. Si bien existen herramientas externas como JSON Schema, no son parte inherente del formato.
- **No Soporta Comentarios:** El formato JSON no permite incluir comentarios dentro del archivo. Esto puede dificultar la documentación del propósito de ciertos campos o estructuras directamente en el archivo JSON, obligando a los desarrolladores a mantener la documentación por separado.

- **Limitaciones en Datos Binarios:** JSON está diseñado para datos de texto. Si necesitas incluir datos binarios (como imágenes o archivos de audio) directamente en un JSON, estos deben ser codificados primero (por ejemplo, en Base64), lo que aumenta el tamaño del archivo y el consumo de recursos para codificar/decodificar.
- **Menos Extensible que XML en Algunos Casos:** Para escenarios donde se requiere una validación de esquema compleja, atributos de elementos o la capacidad de definir namespaces (espacios de nombres), XML puede ser más potente. Sin embargo, para la mayoría de los casos de uso de intercambio de datos, JSON es más que suficiente y más simple.

2.3 DISEÑO WEB

2.3.1 Fundamentos del Diseño Web

2.3.1.1 Tipografía

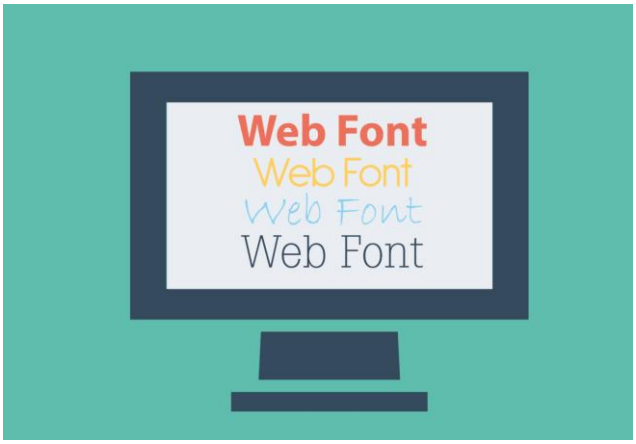


Figura 2. 11 Ejemplos de Tipografía

Si bien el diseño Web sigue básicamente sus propias reglas en cierto modo independientes de otras disciplinas del diseño gráfico, en lo tocante a las tipografías, fuentes o tipos de letra, se ciñe a la regla principal de que todo el contenido debe ser suficientemente legible. Esta es sin duda una regla de oro en cualquier tipo de diseño ya que, sí algo no se puede leer, carece de todo tipo de utilidad. Básicamente, en el diseño Web definiremos dos tipos o grupos diferenciados de textos, los títulos y el cuerpo de texto o mensaje que deseamos transmitir.

En el primero de los casos, será conveniente que utilicemos un tipo de letra grueso y bien detallado. Deberíamos buscar que las tipografías seleccionadas para nuestros títulos concuerden a la perfección con el diseño global de nuestras páginas Web. También tendremos que prestar especial atención al color, ya que, en el caso de los títulos, tiene una componente de significado de gran importancia como los ejemplos de las figuras 2.11 y 2.12.

Para el cuerpo de texto emplearemos tipografías conocidas como "sin Serifa" (por ejemplo, Arial o Verdana), es decir, tipos de letra carentes de cuñas, salientes y rebordes que sirven para adornar las letras en otras tipografías (como Times New Roman). El tipo de letra con el que está escrito el cuerpo de texto de este libro es Avenir, un tipo de letra "sin Serifa". Un ejemplo de tipo de letra "con Serifa" sería el siguiente: Times New Roman.

Las tipografías lisas se leen mucho mejor en la pantalla del ordenador gracias a sus formas más rectas. Por el contrario, las tipografías “con Serifa" tienen muchos detalles que, a tamaños pequeños, son difíciles de distinguir en una pantalla, no así sobre el papel escrito. De la mano de la tecnología Flash, surgieron las tipografías conocidas como Pixel Fonts, unos tipos de letra especialmente diseñados para verse bien con un determinado



Figura 2. 12 Otro ejemplo de Tipografías

tamaño que encaja con el tamaño de los píxeles de un monitor. Estas fuentes han tenido gran aceptación y es recomendable utilizarlas para el desarrollo de animaciones Flash, ya que hacen que los textos resulten muy legibles.

2.3.1.2 Color

Algunas de las consideraciones que debería tener en cuenta a la hora de elegir los colores de su sitio Web son las siguientes:

- **Expansión.** Es recomendable utilizar un fondo claro en nuestras páginas Web, debido al carácter expansivo de los colores. Un texto sobre fondo claro es más legible y el lector se encontrará más cómodo con su lectura y no tendrá que forzar la vista.
- **Armonía.** Cree una gama de colores para su sitio Web que esté compuesto por colores de la misma gama o tono.
- **Contraste.** Combine diferentes colores para crear contraste, utilizando claros y oscuros, tonos cálidos y fríos, etc.
- **Significado de los colores.** Cada color tiene un significado que, aunque varía ligeramente de unas culturas a otras, generalmente es reconocido por el tipo de sentimiento o emoción que transmite. Algunos ejemplos son:
  - ✓ **Blanco.** Potencia los colores que le rodean. Expresa paz, luz, felicidad, actividad, pureza e inocencia.
  - ✓ **Azul.** Indica profundidad, frío. Expresa cierta sensación de placidez distinta al de las tonalidades verdes. Expresa armonía, fidelidad, serenidad, sosiego y sugiere optimismo. No obstante, cuando más claro se hace pierde atracción y se vuelve indiferente.
  - ✓ **Violeta.** Es el color de la templanza, la lucidez y la reflexión. Místico, melancólico e introvertido.
  - ✓ **Verde.** Color tranquilo y sedante. Evoca la vegetación y el frescor de la naturaleza.
  - ✓ **Marrón.** Se trata de un color masculino, severo y confortable. Evoca ambientes otoñales y transmite una impresión de gravedad y equilibrio.
  - ✓ **Negro.** Simboliza silencio, misterio y a veces, lo impuro y maligno. Cuando tiene tonos brillantes confiere nobleza y elegancia.
  - ✓ **Gris.** Simboliza indecisión, ausencia de energía, duda y melancolía. Si se trata de colores metálicos hacen que una imagen sea lustrosa, dan impresión de frialdad y de brillantez, de lujo y elegancia.
  - ✓ **Amarillo.** Es un color luminoso, cálido, ardiente. Es el color del sol, la luz y el oro. Es animado, jovial, excitante, afectivo e impulsivo.
  - ✓ **Naranja.** Es fuerte, activo y radiante. Tiene un carácter acogedor, cálido, estimulante y una cualidad dinámica muy positiva y enérgica.

2.3.1.3 Composición

Es importante que pensemos en la forma en que vamos a estructurar nuestra Web, qué colores vamos a utilizar en cada sitio y por qué, cómo vamos a espaciar nuestros textos, etc.

El texto no debería estar demasiado pegado a los elementos gráficos, ni tampoco tener un interlineado bajo, es decir, pegar demasiado unas líneas a otro. En caso contrario, el texto se volverá confuso para su lectura. El espaciado entre líneas debería ser igual o superior a un 20 por ciento del tamaño de letra. Es decir, para una letra de un tamaño de 10 puntos, el interlineado debería ser de 12 puntos.

La posición de los elementos que componen nuestra página es uno de los aspectos más importantes a la hora de llevar a cabo un diseño. No es lo mismo ubicar una imagen arriba o abajo, con un texto al lado o sin él, que tenga un tamaño grande o pequeño, y un largo etcétera. Una imagen grande servirá para mostrar algún aspecto de nuestra página Web que deseemos destacar.

Luego, podremos añadir imágenes pequeñas para embellecer y dotar de detalles las partes que tienen más texto. La colocación de imágenes pequeñas cerca del texto cumple dos funciones: que el usuario tenga una información visual de lo que va a leer, atrayendo su atención hacia donde nosotros queramos y hacer más atractiva la página.

Cualquier usuario se aburrirá ante una página llena de texto y el resultado será, con toda seguridad, que cambie de página o incluso de sitio Web.

2.3.1.4 Otras consideraciones

El éxito del diseño de un sitio Web dependerá en gran medida de nuestra pericia a la hora de manejar los criterios artísticos y estéticos genéricos de cualquier disciplina del diseño y específicos del medio concreto en el que deseamos transmitir nuestras ideas, la Web. Aparte de lo ya mencionado, algunas consideraciones adicionales para el diseño de nuestros sitios Web serían las siguientes:

- **El peso de las páginas.** Debemos mantener limitado el peso (tamaño de los archivos) de nuestras páginas Web para que su carga no resulte demasiado lenta. ¿Cuál es el tamaño adecuado? Haga pruebas y depure su material hasta que su tiempo de carga sea el menor posible. Un usuario no esperará 30 segundos hasta que aparezca la información de una página. Realice sus pruebas con la conexión más lenta que pueda.
- **Tratamiento de Imágenes.** Para no aumentar artificialmente el tamaño de nuestras páginas, debemos realizar un tratamiento previo a nuestras imágenes para reducir al máximo su tamaño manteniendo a la vez una calidad aceptable.
- **Consistencia.** Debemos lograr que nuestros sitios Web en su globalidad presenten una coherencia y un aspecto que le confiera un carácter distintivo. En esta labor, nos ayudará la utilización de hojas de estilo CSS3, de las que hablaremos más adelante en este libro. Para mantener la consistencia en un

sitio Web tenga en cuenta la utilización de gráficos del mismo estilo en sus páginas, fondos y colores uniformes, familias de fuentes que representen los distintos tipos de texto que utilizaremos en cada página, enlaces y barras de navegación que tengan el mismo aspecto y se sitúen en los mismos lugares, etc.

2.3.1.5 Diseño Web Adaptativo

Un sitio Web construido bajo los criterios del diseño Web adaptativo utiliza consultas medía de CSS3, una extensión de la regla ©*media* para adaptar el diseño al entorno en el que se visualizan las páginas, junto con la utilización de cuadrículas fluidas basadas en proporciones e imágenes flexibles:

- Las consultas medias de CSS3 permiten que la página utilice diferentes reglas de estilo CSS basadas en las características del dispositivo en el que se está mostrando el sitio Web, principalmente, en la anchura del navegador.
- El concepto de cuadrícula fluida hace que el tamaño de los elementos de la página se exprese en unidades relativas, como porcentajes o puntos em, en lugar de unidades absolutas como píxeles o puntos.
- Las imágenes flexibles también se redimensionan con unidades relativas (hasta un máximo del 100 por ciento), evitando de esta manera que se extiendan fuera del elemento que las contiene.

Otras características que debe seguir cualquier diseño Web adaptativo son las siguientes:

- Seguir las estrategias "primero los dispositivos móviles", "JavaScript discreto" y "Mejoras progresivas". Los navegadores de los teléfonos móviles más básicos no interpretan JavaScript ni las consultas medía, de forma que lo recomendable es empezar por crear un sitio Web básico, mejorarlo a continuación para smartphones y finalmente alcanzar el diseño para PC (en lugar de crear desde el principio un diseño complejo e ir degradándolo paulatinamente hasta adaptarlo a los teléfonos móviles más básicos.
- Mejoras progresivas basadas en el navegador, el dispositivo o la detección de características. Cuando un sitio Web debe soportar los dispositivos móviles más básicos que no disponen de JavaScript, la detección del navegador (agente de usuario) y la detección de dispositivos móviles, son dos de las formas de deducir si determinadas características HTML y CSS serán soportadas (como base de un método progresivo de mejora). Sin embargo, estos métodos no son totalmente fiables.

2.3.1.6 Diseño Web Sensible

Como ya hemos mencionado, en la actualidad los diseños Web deben adaptarse a diferentes dispositivos con diferentes formatos de presentación de sus páginas (teléfonos móviles o smartphones, tabletas o tablets, ordenadores portátiles, ordenadores de sobremesa, etc. Por esta razón, es necesario desarrollar herramientas que eviten que los diseñadores tengan que realizar un diseño diferente para cada dispositivo, de manera que un mismo diseño único se adapte perfectamente (mediante técnicas específicas de diseño y programación) a todos los dispositivos de manera simultánea.

Diseñar un sitio Web diferente para cada plataforma, no sólo conllevaría un desperdicio excesivo de tiempo para el diseñador, sino que incurriría en unos costes prohibitivos para el proyecto de desarrollo de un sitio Web. Un diseño Web sensible eliminará todos estos problemas ofreciendo una nueva capacidad de responder con un diseño adaptado a cada tamaño específico del dispositivo que se utilice: un diseño único que se adapte perfectamente a todos los posibles dispositivos.

2.3.1.7 Importancia de la imagen Web corporativa

Las páginas Web son cada vez más relevantes para el mundo de los negocios, sobre todo, para aquellos que operan por Internet. Por lo tanto, nuestros diseños Web deberán ir asociados a la imagen que desea transmitir nuestro cliente a sus visitantes y tendremos que buscar formas de reconocimiento de marcas a través de logotipos y otras herramientas que resalten sus bondades, personalidad y principios. Además, debemos buscar la originalidad.

Las empresas quieren que su marca sea claramente perceptible. Los visitantes deberían ser capaces de reconocer una marca en cuestión de segundos. Los diseñadores deberán centrarse en la selección de los elementos que complementen a la propia marca o realcen la belleza del logotipo El diseñador deberá centrarse en la representación del propio negocio a través de sus páginas Web.

Cuando nos enfrentemos a la creación de nuestros sitos Web, debemos tener en consideración dos aspectos esenciales respecto al usuario, la usabilidad y el aspecto visual. Ambos conceptos están íntimamente ligados, pues un buen diseño hace que una página resulte fácil de navegar y de interactuar con ella. Además, nuestros diseños deberán tener en cuenta los avances actuales de la industria y estar preparados para los futuros cambios que experimentará la Web a lo largo del tiempo.

2.3.2 Diseño de Páginas Web

2.3.2.1 HTML

El lenguaje HTML (hypertext markup language) se utiliza para crear documentos que muestren una estructura de hipertexto. Un documento de hipertexto es aquel que contiene información cruzada con otros documentos, lo cual nos permite pasar

de un documento al referenciado desde la misma aplicación con la que lo estamos visualizando. HTML permite, además, crear documentos de tipo multimedia, es decir, que contengan información más allá de la simplemente textual como, por ejemplo:

- **Imágenes**
- **Video**
- **Sonido**
- **Subprogramas activos (plug-ins, applets)**

Los documentos HTML se conforman como documentos de texto plano (sin ningún tipo de formateo especial), en los que todo el formato del texto se especifica mediante marcas de texto (llamados etiquetas o tags), que delimitan los contenidos a los que afecta la etiqueta (disponemos de etiquetas de inicio y de final de marcado).

Las **etiquetas** o **tags** son los componentes fundamentales que se usan para **estructurar el contenido** de una página web y darle un significado. Son como los ladrillos que construyen el edificio de tu sitio web, indicándole al navegador cómo debe interpretar y mostrar la información.

La mayoría de las etiquetas HTML se presentan en **pares**, consistiendo en una **etiqueta de apertura** y una **etiqueta de cierre**. La etiqueta de apertura marca el inicio de un elemento, y la de cierre (que incluye una barra inclinada / antes del nombre de la etiqueta) marca su final. Todo el contenido entre estas dos etiquetas es afectado por ellas. Ejemplo:

```
<title>Nombre del documento</title>
<P>Un ejemplo de uso de las etiquetas para marcado de texto</P>
<B>Negrilla<I>Itálica</I>Negrilla</B>
```

Las etiquetas no son sensibles a mayúsculas/minúsculas.

Estructura de los documentos HTML

Todos los documentos HTML siguen aproximadamente la misma estructura. Todo el documento debe ir contenido en una etiqueta HTML, dividiéndose en dos partes: la cabecera, contenida en una etiqueta HEAD y el cuerpo del documento (donde está la información del documento), que está envuelto por una etiqueta BODY. La cabecera contiene algunas definiciones sobre el documento: su título, marcas extra de formato, palabras clave, etc. Ejemplo:

```
<HTML>
    <HEAD>
        <TITLE>Título del documento</TITLE>
    </HEAD>
    <BODY>
        Texto del documento
    </BODY>
</HTML>
```

Si abrimos un documento con este contenido en un navegador apreciaremos que lo que contienen las etiquetas TITLE no se visualiza en el documento, sino que el navegador las visualiza en la barra de título de la ventana.

Bloques de texto

➢ Párrafos

Son los bloques de texto más comunes y se definen con la etiqueta <p>. Esta etiqueta agrupa una serie de oraciones o frases que tratan sobre una misma idea. Los navegadores suelen añadir un espacio en blanco antes y después de cada párrafo para mejorar la legibilidad. Ejemplo:

```
<p>Este es el primer párrafo de mi documento HTML.</p>
<p>Aquí tienes otro párrafo, donde continúo desarrollando mis ideas.</p>
```

➢ Salto de línea

Un **salto de línea** se inserta con la etiqueta **
**. A diferencia de la mayoría de las etiquetas HTML,
 es una **etiqueta vacía** (o de autocierre), lo que significa que no tiene una etiqueta de cierre. Su función es simplemente forzar que el contenido que le sigue comience en una nueva línea, sin crear un nuevo párrafo.Es útil cuando necesitas dividir líneas dentro del mismo bloque de texto, como en una dirección o un poema. Ejemplo:

```
<p>
    Mi dirección es:<br>
    Calle Falsa 123<br>
    Ciudad Inventada, País
</p>
```

➢ Regla horizontal

Una **regla horizontal** se utiliza para crear una línea divisoria temática entre secciones de contenido. Se representa con la etiqueta **<hr>** (del inglés *horizontal rule*). La etiqueta <hr> es una **etiqueta vacía** (o de autocierre), lo que significa que no requiere una etiqueta de cierre. Su función principal es semántica: indicar un cambio de tema o una separación dentro del contenido de una página, haciendo que el documento sea más legible y estructurado.

- **Párrafos citados**

Los bloques citados se utilizan para indicar que un fragmento de texto es una cita larga de otra fuente. Para esto, se usa la etiqueta <blockquote>. Los navegadores suelen aplicar una sangría especial a estos bloques para distinguirlos visualmente del resto del texto. Puedes usar el atributo cite para especificar la URL de la fuente original. Ejemplo:

```
<blockquote>
  <p>La imaginación es más importante que el conocimiento.</p>
  <cite>Albert Einstein</cite>
</blockquote>
```

- **Divisiones**

La etiqueta es un contenedor genérico de "división" o "sección". No tiene ningún significado semántico intrínseco por sí misma, pero es extremadamente útil para agrupar otros elementos HTML y aplicarles estilos con CSS (hojas de estilo en cascada) o manipularlos con JavaScript. Es uno de los pilares del diseño web moderno para organizar el diseño de la página.

```
<div style="background-color: lightblue; padding: 20px;">
  <h2>Acerca de Nosotros</h2>
  <p>Somos una empresa dedicada a...</p>
</div>
```

Enlaces

Una de las características más destacadas y de mayor influencia en el éxito de la web ha sido su carácter hipertexto, es decir, la posibilidad de que podamos enlazar documentos que pueden residir en diferentes servidores de forma intuitiva y transparente. Cabe destacar que los enlaces no sólo llevan a páginas web, sino también a imágenes, audio, vídeo, etc.

Los enlaces se crean utilizando la etiqueta **<a>**, que significa "anchor" (ancla), que con su conjunto de atributos, NAME, HREF, TARGET, nos permite un total control a la hora de crear enlaces en los documentos.

La etiqueta <a> define un hipervínculo. El destino del enlace se especifica mediante el atributo **href** (Hypertext REFerence). El texto o la imagen que se encuentra entre las etiquetas de apertura y cierre de <a> es lo que se convierte en el enlace "clicable". Ejemplo:

```
<a href="URL_de_destino">Texto del enlace o imagen</a>
```

Disponemos de cuatro tipos de enlaces principales:

- **Enlaces a otras páginas web (Absolutos)**

Estos enlaces apuntan a una URL completa, incluyendo el protocolo (http:// o https://) y el dominio. Se utilizan **para enlazar a sitios web externos. Ejemplo:**

```
<a href="https://www.google.com">Visita Google</a>
```

Al hacer clic en "Visita Google", el navegador te llevará a la página principal de Google.

- **Enlaces a páginas dentro del mismo sitio (Relativos)**

Estos enlaces se utilizan para navegar entre páginas de tu propio sitio web. La URL es "relativa" a la ubicación del archivo HTML actual. Esto es muy útil porque no tienes que preocuparte por el dominio si cambias tu sitio de servidor. Ejemplo:

Si tienes una página llamada acerca.html en la misma carpeta que tu página actual:

```
<a href="acerca.html">Acerca de nosotros</a>
```

Si quieres ir a la página principal que está una carpeta arriba de tu ubicación actual:

```
<a href="../index.html">Ir a la página de inicio</a>
```

- **Enlaces a secciones específicas dentro de la misma página (Anclas)**

Puedes enlazar a una sección específica dentro de la misma página web. Esto es útil para tablas de contenido o para saltar a información relevante sin tener que desplazarse. Para esto, necesitas dos partes:

El destino: Crea un "ancla" en la sección a la que quieres enlazar usando el atributo id en cualquier elemento HTML. El id debe ser único en la página.

```
<h2 id="seccion-importante">Esta es una sección importante</h2>
<p>Contenido de la sección...</p>
```

El enlace: Utiliza la etiqueta <a> con el atributo href apuntando al id del destino, precedido por un #.

```
<a href="#seccion-importante">Ir a la sección importante</a>
```

- **Enlaces para enviar correos electrónicos**

Puedes crear enlaces que abran el cliente de correo electrónico predeterminado del usuario para enviar un email. Usa mailto: en el atributo href.

```
<a href="mailto:tuemail@ejemplo.com">Enviar un correo electrónico</a>
```

Listas

Las **listas** en HTML son una forma fundamental de organizar la información de manera estructurada y legible. Permiten presentar elementos de forma secuencial,

ya sea con un orden específico o sin él. HTML ofrece principalmente tres tipos de listas:

1. **Listas No Ordenadas (<ul>)**
   Las listas no ordenadas se utilizan para agrupar elementos donde el orden de los ítems no es importante. Cada ítem de la lista se marca con un punto (un "bullet point") por defecto.

   - Se definen con la etiqueta <ul> (unordered list).
   - Cada ítem dentro de la lista se define con la etiqueta <li> (list item).

   Ejemplo:

```
<ul>
  <li>Manzanas</li>
  <li>Naranjas</li>
  <li>Plátanos</li>
</ul>
```

2. **Listas Ordenadas (<ol>)**
   Las listas ordenadas se utilizan para agrupar elementos donde el orden de los ítems sí es importante, como pasos en una receta, un ranking o una secuencia de instrucciones. Cada ítem de la lista se marca con un número (por defecto), una letra o un número romano.

   - Se definen con la etiqueta <ol> (ordered list).
   - Cada ítem dentro de la lista se define con la etiqueta <li> (list item).

   Ejemplo:

```
<ol>
  <li>Paso 1: Preparar los ingredientes</li>
  <li>Paso 2: Mezclar todo</li>
  <li>Paso 3: Hornear a 180°C</li>
</ol>
```

3. **Listas de Definición**
   Las listas de definición se utilizan para presentar una lista de términos y sus correspondientes descripciones o definiciones. Son ideales para glosarios, preguntas frecuentes o cualquier contenido donde necesites emparejar un término con su explicación.

   - Se define con la etiqueta <dl> (description list).
   - Cada término se define con la etiqueta <dt> (description term).
   - Cada descripción del término se define con la etiqueta <dd> (description description).

   Ejemplo:

```
<dl>
  <dt>HTML</dt>
  <dd>Lenguaje de marcado para la creación de páginas web.</dd>

  <dt>CSS</dt>
  <dd>Lenguaje de hojas de estilo utilizado para describir la presentación
de un documento HTML.</dd>

  <dt>JavaScript</dt>
  <dd>Lenguaje de programación que permite añadir interactividad a las
páginas web.</dd>
</dl>
```

Imágenes

Las imágenes son un componente visual esencial en cualquier página web, ya que enriquecen el contenido y lo hacen más atractivo. Disponemos de una única etiqueta: <img>.

Esta etiqueta dispone de diversos atributos que nos permiten especificar qué fichero de imagen queremos usar, las medidas de ésta, etc. El atributo que nos permite especificar qué imagen debemos mostrar es **SRC**. Las imágenes que referenciemos con el atributo SRC pueden estar en cualquier directorio del servidor, en otros servidores, etc. El valor que pasemos a SRC debe ser una URL.

Disponemos, además, de un atributo ALT que nos permite asignar un texto alternativo a la imagen para el caso de que el navegador no pudiese mostrar la imagen que enseñase ese texto al usuario. Ejemplo:

```
<img src="perro-jugando.jpg" alt="Un perro labrador feliz jugando con una
pelota en un parque.">
```

Tablas

Las tablas en HTML se utilizan para mostrar datos tabulares, es decir, información organizada en filas y columnas. Son ideales para presentar datos estructurados como estadísticas, horarios, listas de precios, etc.

Estructura básica de una tabla

Una tabla en HTML se construye con varias etiquetas anidadas:

- **<table>:** La etiqueta principal que define el inicio y el fin de la tabla.
- **<tr> (table row):** Define una **fila** dentro de la tabla.
- **<th> (table header):** Define una **celda de encabezado** (header cell). Estas celdas suelen contener los títulos de las columnas o filas y el navegador las muestra en negrita y centradas por defecto.

- **<td> (table data):** Define una **celda de datos** (data cell), que contiene el contenido de la tabla.

Ejemplo:

```
<table>
  <tr>
    <th>Nombre</th>
    <th>Edad</th>
    <th>Ciudad</th>
  </tr>
  <tr>
    <td>Juan</td>
    <td>30</td>
    <td>Madrid</td>
  </tr>
  <tr>
    <td>María</td>
    <td>25</td>
    <td>Barcelona</td>
  </tr>
</table>
```

Formularios

Los formularios son elementos fundamentales en HTML que permiten a los usuarios interactuar con una página web, enviar datos a un servidor y realizar diversas acciones, como iniciar sesión, registrarse, realizar búsquedas, enviar comentarios, etc. En esencia, son la interfaz a través de la cual los usuarios proveen información.

Un formulario HTML se define mediante la etiqueta **<form>**. Esta etiqueta actúa como un contenedor para todos los elementos de entrada (inputs), botones y otros controles que recopilan la información del usuario.

Los atributos más importantes de la etiqueta <form> son:

- **action**: Especifica la **URL** a la que se enviarán los datos del formulario cuando este sea enviado. Normalmente, esta URL apunta a un *script* en el servidor (por ejemplo, un archivo PHP, Python, Node.js) que procesará la información.

```
<form action="/procesar_formulario.php">

</form>
```

- **method**: Define el **método HTTP** que se utilizará para enviar los datos del formulario. Los dos valores más comunes son:
    - **GET**: Los datos del formulario se envían como parte de la URL (aparecen en la barra de direcciones del navegador como parámetros

de consulta). Es adecuado para formularios que solo recuperan datos (ej., búsquedas) y para datos que no son sensibles ni muy largos.

- **POST**: Los datos del formulario se envían en el cuerpo de la solicitud HTTP. Es el método preferido para enviar datos sensibles (contraseñas, información personal) o grandes cantidades de datos, ya que no son visibles en la URL.

```
<form action="/enviar_datos_seguros" method="post">

</form>
```

- **enctype**: Se utiliza cuando el formulario va a enviar archivos (como imágenes o documentos). El valor más común es multipart/form-data. Solo se usa con method="post".

```
<form action="/subir_archivo" method="post" enctype="multipart/form-data">
    <input type="file" name="documento">
    <button type="submit">Subir</button>
</form>
```

Elementos del formulario

HTML proporciona una gran variedad de elementos de entrada para los formularios. Estos elementos permiten desde introducir texto hasta enviar ficheros.

1. Campos de Entrada (<input>)

El elemento INPUT es quizás el más conocido y usado de los elementos de los formularios. Actúa como un campo de entrada, disponiendo de diversos tipos de elementos INPUT en función del valor del atributo TYPE:

- **TYPE=RADIO:** Permite escoger entre múltiples opciones, pero solo una de las del mismo nombre puede ser seleccionada a la vez.
- **TYPE=RESET:** Restablece todo el formulario a sus valores iniciales, borrando cualquier entrada del usuario.
- **TYPE=TEXT:** Permite la entrada de una sola línea de texto libre.
- **TYPE=PASSWORD:** Permite la entrada de una sola línea de texto, mostrando caracteres como asteriscos (*) en lugar del texto real, usado comúnmente para contraseñas.
- **TYPE=CHECKBOX:** Permite escoger una o varias opciones de un conjunto de múltiples posibilidades.
- **TYPE=SUBMIT:** Crea un botón que, al ser clicado, envía los datos introducidos en el formulario y ejecuta la acción especificada.
- **TYPE=HIDDEN:** Crea un campo de texto que no es visible para el usuario. Se utiliza para conservar y enviar valores que no necesitan ser mostrados o modificados por el usuario.
- **NAME:** Asigna un nombre al campo. Este nombre es crucial para poder identificar y procesar los datos de ese campo desde programas o scripts del lado del servidor.
- **VALUE:** Otorga un valor inicial o predeterminado al campo.

- **SIZE:** Define el tamaño visible del campo, específicamente para los tipos TEXT y PASSWORD.
- **MAXLENGTH:** Establece la longitud máxima de caracteres que el usuario puede introducir en los campos TEXT y PASSWORD.
- **CHECKED:** Se utiliza en los campos RADIO o CHECKBOX para indicar si la opción correspondiente está marcada por defecto cuando el formulario se carga.

2. Elemento SELECT

El elemento SELECT nos permite seleccionar una o varias de las opciones presentadas. Un ejemplo de elemento SELECT sería:

```
<SELECT name="destino">
 <option> África
 <option> Antártida
 <option> América
 <option> Asia
 <option> Europa
 <option> Oceanía
</SELECT>
```

3. Elemento TEXTAREA

El elemento TEXTAREA nos permite recoger del usuario elementos de texto de múltiples líneas. El formato es el siguiente:

```
<TEXTAREA name="comentarios" cols=30 rows=6>
    Introduzca comentarios sobre la página
</TEXTAREA>
```

2.3.2.2 HTML avanzado

Cuando hablamos de HTML avanzado, a menudo nos referimos a cómo HTML trabaja en conjunto con otras tecnologías web para crear sitios más complejos y visualmente atractivos. Las Hojas de Estilo en Cascada (CSS) son una de las tecnologías más cruciales que complementan a HTML.

Mientras que HTML se encarga de la estructura y el contenido de una página web (qué elementos hay y qué significan), CSS se encarga de la presentación y el estilo (cómo se ven esos elementos).

Hojas de Estilo

CSS es un lenguaje de diseño que se utiliza para describir la presentación de un documento escrito en HTML (o XML). Con CSS, puedes controlar una amplia gama de aspectos visuales de tu página web, incluyendo:

- **Colores:** del texto, fondos, bordes.

- **Tipografías:** tipo de fuente, tamaño, peso (negrita), estilo (cursiva).
- **Diseño (Layout):** cómo se posicionan los elementos en la página (márgenes, rellenos, alineación, flotación, columnas, cuadrículas).
- **Espaciado:** entre elementos y dentro de ellos.
- **Fondos:** imágenes de fondo, colores de fondo.
- **Bordes:** estilo, grosor y color de los bordes.
- **Efectos visuales:** sombras, transiciones, animaciones.
- **Responsividad:** cómo se ve el sitio en diferentes tamaños de pantalla (móviles, tabletas, escritorios).

Existen tres formas principales de aplicar CSS a un documento HTML:

1. **CSS Externo (Recomendado):**
   - Los estilos se definen en un archivo .css separado.
   - Este archivo se vincula al documento HTML usando la etiqueta <link> dentro de la sección <head>.
   - Es el método más común y recomendado por sus beneficios de mantenimiento y eficiencia.

```
<head>
    <link rel="stylesheet" href="estilos.css">
</head>
```

2. **CSS Interno:**
   - Los estilos se definen directamente dentro de la sección <head> del documento HTML, utilizando la etiqueta <style>.
   - Útil para estilos específicos de una sola página o para pruebas, pero no es ideal para sitios grandes.

```
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: lightblue;
        }
        h1 {
            color: navy;
        }
    </style>
</head>
```

3. **CSS en Línea (Evitar en la mayoría de los casos):**
    - Los estilos se aplican directamente a un elemento HTML individual utilizando el atributo style.
    - Es la forma menos recomendada porque mezcla el contenido con la presentación, dificulta el mantenimiento y la consistencia.
    - Solo debe usarse para pequeñas modificaciones muy específicas que no se repetirán.

```
<p style="color: red; font-size: 18px;">Este texto es rojo y grande.</p>
```

2.3.2.3 HTML Dinámico

Se refiere a las páginas web cuyo contenido o apariencia pueden cambiar después de que la página se ha cargado inicialmente en el navegador, sin necesidad de recargarla completamente. A diferencia del HTML "estático" tradicional, que se muestra exactamente como fue creado, el HTML dinámico ofrece interactividad y una experiencia de usuario más rica y responsiva.

El HTML por sí mismo es un lenguaje de marcado estático. Para que se vuelva dinámico, necesita la ayuda de otras tecnologías, principalmente:

1. **JavaScript:** Es el lenguaje de programación estándar para la web. JavaScript permite:
    - **Manipular el DOM (Document Object Model):** El DOM es una representación en árbol de la estructura de un documento HTML. JavaScript puede añadir, modificar o eliminar elementos HTML, atributos y estilos en tiempo real.
    - **Manejar eventos:** Responder a acciones del usuario como clics de ratón, pulsaciones de teclado, envío de formularios, etc.
    - **Realizar peticiones al servidor (AJAX/Fetch API):** Obtener o enviar datos desde/hacia un servidor sin recargar la página, lo que permite actualizar partes del contenido de forma asíncrona.
    - **Validar formularios:** Comprobar la entrada del usuario antes de enviarla.
    - **Crear animaciones y efectos visuales.**
2. **CSS (Hojas de Estilo en Cascada):** Aunque CSS define el estilo, combinado con JavaScript, puede usarse dinámicamente para:
    - **Cambiar estilos:** JavaScript puede añadir o eliminar clases CSS, o modificar propiedades de estilo directamente, alterando la apariencia de los elementos.
    - **Transiciones y animaciones:** CSS permite crear animaciones complejas que pueden ser disparadas por JavaScript.
3. **Tecnologías del lado del servidor:** Aunque la "dinamicidad" de la que hablamos aquí ocurre en el navegador del usuario, es común que el HTML dinámico se alimente de datos generados o procesados por lenguajes del lado del servidor como Python (con Django/Flask), Node.js, PHP, Ruby (con Rails),

etc. Estos lenguajes generan el HTML inicial que luego JavaScript hace dinámico.

Ejemplos de HTML Dinámico

Casi todas las interacciones modernas en la web utilizan HTML dinámico:

- **Menús desplegables:** Un menú que se abre o cierra al hacer clic.
- **Pestañas (Tabs):** Contenido que cambia al seleccionar una pestaña sin recargar la página.
- **Validación de formularios en tiempo real:** Ver un mensaje de error inmediatamente después de escribir un email con formato incorrecto.
- **Resultados de búsqueda instantáneos:** Sugerencias que aparecen mientras escribes en una barra de búsqueda (como Google).
- **Galerías de imágenes:** Navegar por imágenes sin recargar la página.
- **Carga de más contenido (infinite scroll):** Cuando llegas al final de una página y se carga automáticamente más contenido.
- **Aplicaciones de una sola página (SPAs):** Aplicaciones web completas como Gmail o Google Docs, donde la mayor parte de la navegación y el contenido se cargan dinámicamente sin recargar la página entera.



Figura 2. 13 Ejemplo de uso de un HTML estático y un dinámico

2.3.2.4 JavaScript

Javascript es un lenguaje de programación interpretado (un lenguaje de tipo script). A pesar de que existen intérpretes no dependientes de ningún navegador, es un lenguaje de script que suele encontrarse vinculado a páginas web. Javascript y Java son dos lenguajes de programación distintos con filosofías muy diferentes. El único punto en común es la sintaxis, ya que cuando Netscape diseñó Javascript, se inspiró en la sintaxis de Java.

El primer programa sencillo

Como resulta habitual al mostrar lenguajes de programación, nuestro primer contacto con Javascript será realizar un primer programa que nos muestre el ya clásico mensaje "Hola mundo". Como Javascript es un lenguaje que veremos, generalmente, vinculado a una página web, el siguiente código será un fichero HTML que deberemos visualizar en un navegador.

```
<HTML>

<HEAD>
    <SCRIPT LANGUAGE="Javascript">
        function Saludo() {
            alert("Hola mundo");
        }
    </SCRIPT>
</HEAD>

<BODY>
    <FORM>
        <INPUT TYPE="button" NAME="Boton" VALUE="Pulsar" onClick="Saludo()">
    </FORM>
</BODY>

</HTML>
```

Este programa Javascript pinta en nuestra pantalla un botón. Al pulsarlo, se abre una ventana con un mensaje. El aspecto que ofrece este programa es el siguiente:

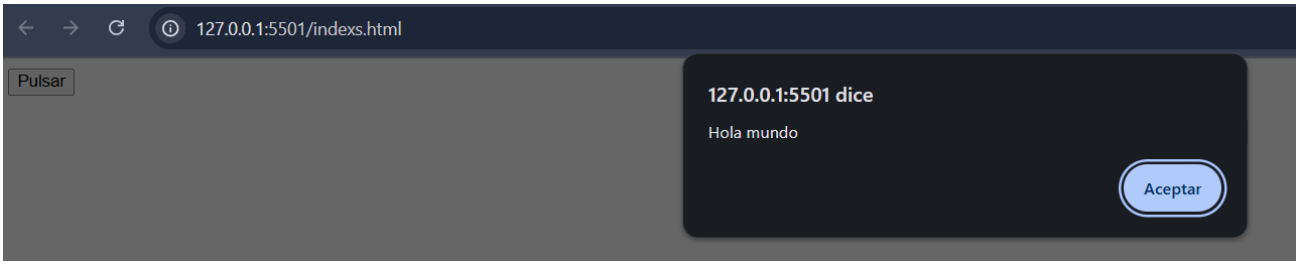


Figura 2. 14 Ejemplo del funcionamiento de JavaScript con el mensaje "Hola mundo"

Elementos básicos de JavaScript

Los programas de JavaScript se construyen a partir de elementos fundamentales:

- **Sentencias (Statements):** Son las instrucciones que ejecuta el programa. Cada sentencia suele terminar con un punto y coma (;), aunque a veces es opcional.
- **Comentarios:** Líneas de código que el intérprete ignora, utilizadas para documentar o explicar el código.
    - Comentarios de una línea: // Este es un comentario
    - Comentarios de varias líneas: /* Esto es un comentario multi-línea */
- **Identificadores:** Nombres que se dan a variables, funciones, etc. Deben comenzar con una letra, guion bajo (_) o signo de dólar ($), y pueden contener números. Son sensibles a mayúsculas y minúsculas (nombre es diferente de Nombre).
- **Palabras reservadas (Keywords):** Palabras que tienen un significado especial en JavaScript y no pueden usarse como identificadores (ej., function, if, var, const, let).
- **Operadores:** Símbolos que realizan operaciones sobre uno o más valores (operandos). Incluyen operadores aritméticos (+, -, *, /), de asignación (=, +=), de comparación (==, ===, >, <), lógicos (&&, ||, !), etc.

Tipos de datos y variables

JavaScript maneja diferentes tipos de datos y utiliza **variables** para almacenar y manipular estos datos.

Variables: Son contenedores para almacenar valores. Se declaran usando var, let o const.

- **var:** Declaración antigua, con ámbito de función (puede generar comportamientos inesperados).
- **let:** Declaración moderna, con ámbito de bloque (recomendado para variables que cambian).
- **const:** Declaración moderna, para constantes (variables cuyo valor no cambia después de la asignación inicial).

Tipos de Datos Primitivos:

- **string (cadena de texto):** Secuencias de caracteres (ej., "Hola mundo", 'JavaScript').
- **number (número):** Números enteros o decimales (ej., 10, 3.14). JavaScript no distingue entre enteros y flotantes.
- **boolean (booleano):** Representa un valor de verdad: true (verdadero) o false (falso).

- **undefined:** Una variable que ha sido declarada pero aún no se le ha asignado un valor.
- **null:** Representa la ausencia intencionada de cualquier valor u objeto. Es un valor asignado.
- **symbol (símbolo):** Un tipo de dato único e inmutable (introducido en ES6).
- **bigint (entero grande):** Para números enteros de precisión arbitraria (muy grandes).

Tipo de Datos Complejo:

- **object (objeto):** Colección de propiedades (clave-valor). Incluye objetos genéricos, arrays, funciones, etc.

Estructuras de control

Las estructuras de control determinan el flujo de ejecución de un programa, permitiendo tomar decisiones y repetir acciones.

- **Condicionales (if, else if, else):** Ejecutan bloques de código si se cumple una condición.

```
let puntuacion = 85;
if (puntuacion >= 90) {
    console.log("Excelente.");
} else if (puntuacion >= 70) {
    console.log("Bueno."); // Este se ejecuta
} else {
    console.log("Necesitas mejorar.");
}
// Salida: Bueno.
```

- **Bucle switch:** Evalúa una expresión y ejecuta bloques de código diferentes según el valor de esa expresión.

```
let fruta = "manzana";
switch (fruta) {
    case "platano":
        console.log("Es un plátano.");
        break;
    case "manzana":
        console.log("Es una manzana."); // Este se ejecuta
        break;
    default:
        console.log("No sé qué fruta es.");
}
// Salida: Es una manzana.
```

- **Bucles (Loops):** Repiten un bloque de código varias veces.
  - **for:** Repite un bloque de código un número específico de veces.

```
// Contar hasta 3
for (let i = 1; i <= 3; i++) {
    console.log(i);
}
// Salida:
// 1
// 2
// 3
```

  - **while:** Repite un bloque de código mientras una condición sea verdadera.

```
let i = 0;
while (i < 3) {
    console.log("Hola");
    i++;
}
// Salida:
// Hola
// Hola
// Hola
```

  - **do...while:** Similar a while, pero garantiza que el bloque de código se ejecute al menos una vez antes de verificar la condición.

```
let num = 5;
do {
    console.log("El número es " + num);
    num++;
} while (num < 5); // La condición es falsa, pero se ejecuta una vez
// Salida:
// El número es 5
```

  - **for...in:** Itera sobre las propiedades enumerables de un objeto.

```
const usuario = { nombre: "Leo", edad: 30 };
for (const propiedad in usuario) {
    console.log(propiedad + ": " + usuario[propiedad]);
}
// Salida:
// nombre: Leo
// edad: 30
```

- **for...of:** Itera sobre los valores de objetos iterables (como arrays, strings).

```
const colores = ["rojo", "azul"];
for (const color of colores) {
    console.log(color);
}
// Salida:
// rojo
// azul
```

Funciones

Una función es un bloque de código fuente que contiene un conjunto de instrucciones y que puede ser utilizada desde el código fuente que escribes tantas veces como necesites.

Las funciones tienen la capacidad de:

- Recibir datos de entrada para su ejecución.
- Devolver datos como resultado de la ejecución.

Ambas capacidades son opcionales, es decir, puedes tener funciones que no reciben datos y que no devuelven nada, funciones que reciben datos y que no devuelven nada, funciones que no reciben datos y que devuelven datos y por último funciones que reciben datos y que devuelven datos.

La utilización de funciones es beneficiosa ya que aporta las siguientes características al código fuente:

- Simplificación del código.
- Mejor organización del código.
- Reutilización de código fuente.

La sintaxis de las funciones en JavaScript es la siguiente:

```
function NombreFuncion (parámetros){
    BloqueInstrucciones;
    return ValorRetorno;
}
```

Veamos los elementos en detalle:

- **function:** palabra utilizada para definir una función en JavaScript.
- **NombreFuncion:** nombre que tendrá la función. Te aconsejamos que utilices nombres de funciones descriptivos que representen lo que la función hace.
- **parámetros:** conjunto de elementos de entrada que tiene la función. Los parámetros son opcionales, es decir, puede haber 0 o más. En caso de ser más de uno los parámetros, irán separados por coma.

- **BloqueInstrucciones:** bloque de código que ejecuta la función.
- **return:** retorna datos al código fuente que utilizó la función. Es opcional, ya que el retorno de datos no es obligatorio en las funciones.
- **ValorRetorno:** datos que se retornan.

Hasta aquí te hemos explicado cómo se definen funciones en JavaScript. Para poder utilizar funciones en JavaScript desde el código fuente tienes que hacerlo de la siguiente manera:

```
Variable = NombreFuncion(parámetros);
```

Veamos los elementos en detalle:

- **Variable:** almacenará el valor que devuelve la función. Es opcional, ya que se suprime si la función no devuelve nada.
- **NombreFuncion:** nombre de la función que vamos a utilizar.
- **Parámetros:** parámetros de entrada que tiene la función y que se escriben separados por comas en caso de ser más de uno. Es opcional, por lo que, si la función no recibe parámetros, se suprimirá.

Eventos

En JavaScript, los eventos son acciones o sucesos que ocurren en el navegador o en la página web. Piensa en ellos como señales que el navegador emite y a las que tu código JavaScript puede "escuchar" y "responder". Los eventos son la clave para hacer que las páginas web sean interactivas y dinámicas. Vea la tabla 1 para conocer algunos eventos de Javascript.

Tabla 2.1 Eventos más comunes en Javascript

Evento	Descripción
click	Se produce cuando el usuario hace clic en un elemento.
mouseover	Se produce cuando el puntero del mouse se mueve sobre un elemento.
mouseout	Se produce cuando el puntero del mouse sale de un elemento.
keydown	Se produce cuando el usuario presiona una tecla.
keyup	Se produce cuando el usuario libera una tecla.
submit	Se produce cuando se envía un formulario.
change	Se produce cuando el valor de un elemento (como un campo de entrada) cambia.
load	Se produce cuando un objeto (por ejemplo, una imagen) ha terminado de cargar.
resize	Se produce cuando se cambia el tamaño de la ventana del navegador.
scroll	Se produce cuando el usuario desplaza una barra de desplazamiento.
dblclick	Se produce cuando el usuario hace doble clic en un elemento.
focus	Se produce cuando un elemento recibe el foco (por ejemplo, al hacer clic en un campo de entrada).
blur	Se produce cuando un elemento pierde el foco.
contextmenu	Se produce cuando el usuario hace clic derecho en un elemento.
dragstart	Se produce cuando el usuario comienza a arrastrar un elemento.
dragend	Se produce cuando el usuario termina de arrastrar un elemento.
drop	Se produce cuando el usuario suelta un elemento arrastrado en un destino.
input	Se produce cuando el valor de un elemento <input>, <select> o <textarea> cambia (similar a change, pero se dispara más frecuentemente).

Capítulo III

Sitio Web de la Escuela de Posgrados de Excelencia

En este capítulo se detalla el proceso técnico para el desarrollo e implementación del proyecto, que busca unificar los cinco campus de EP de México en una única plataforma web. Se describirá la construcción del **Frontend**, la creación de un panel de administración en el **Backend** y la interconexión de ambas partes a través de un archivo **JSON**, siguiendo un enfoque secuencial y replicable.

3.1. CONFIGURACION DEL ENTORNO DE DESARROLLO

Para llevar a cabo el proyecto, se utilizó un entorno de desarrollo integrado (IDE) y un editor de código que facilitaron la gestión del código, la depuración y la colaboración. La elección de estas herramientas se basó en su compatibilidad con las tecnologías del proyecto y su robusta suite de funcionalidades. A continuación, se detallan las herramientas y la estructura de archivos que sirvieron como base para el desarrollo.

Herramientas de desarrollo

- **Editor de código (Frontend):** Para el desarrollo de la interfaz de usuario (Frontend), se utilizó Visual Studio Code (VS Code). Esta elección se debió a su amplia gama de extensiones, como Live Server para la visualización en tiempo real y ESLint para el análisis estático del código, lo que permitió mantener un estilo de codificación consistente y detectar errores de manera temprana.

- **IDE (Backend):** Para el desarrollo de la lógica del servidor (Backend), se optó por Visual Studio 2022 Community. Este entorno de desarrollo integrado (IDE) fue la elección principal debido a su robusto soporte para proyectos ASP.NET Web Form. Visual Studio fue fundamental para la gestión del proyecto, ya que ofrece herramientas integradas para la depuración avanzada, la gestión de bases de datos y la refactorización de código, lo que optimizó significativamente el proceso de desarrollo del servidor. Su integración con las tecnologías de Microsoft permitió un entorno de trabajo unificado y eficiente.

Estructura del proyecto

El proyecto se organizó en una estructura de carpetas modular y coherente para mantener una clara separación de responsabilidades entre el Frontend y el Backend, así como para facilitar la colaboración y el mantenimiento a largo plazo. La estructura principal del proyecto se ilustra a continuación, separando las tecnologías de manera lógica véase figuras 3.1 y 3.2.

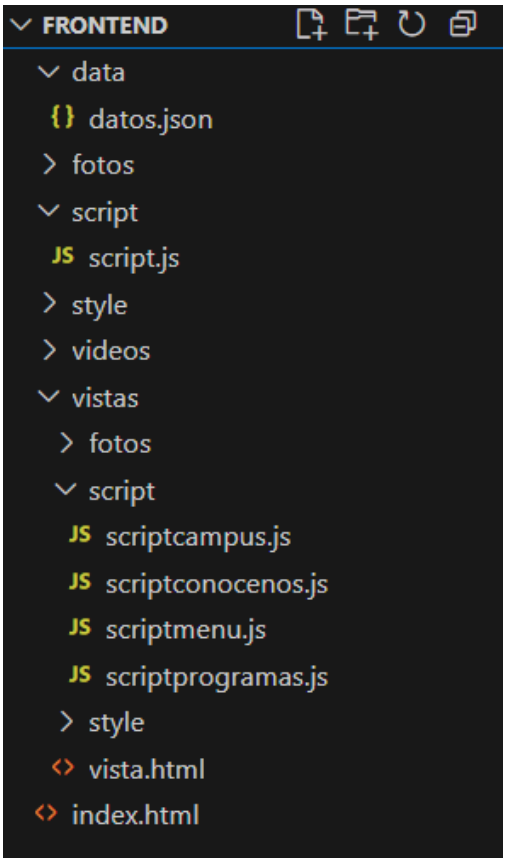


Figura 3. 2 Estructura del Proyecto (Frontend)

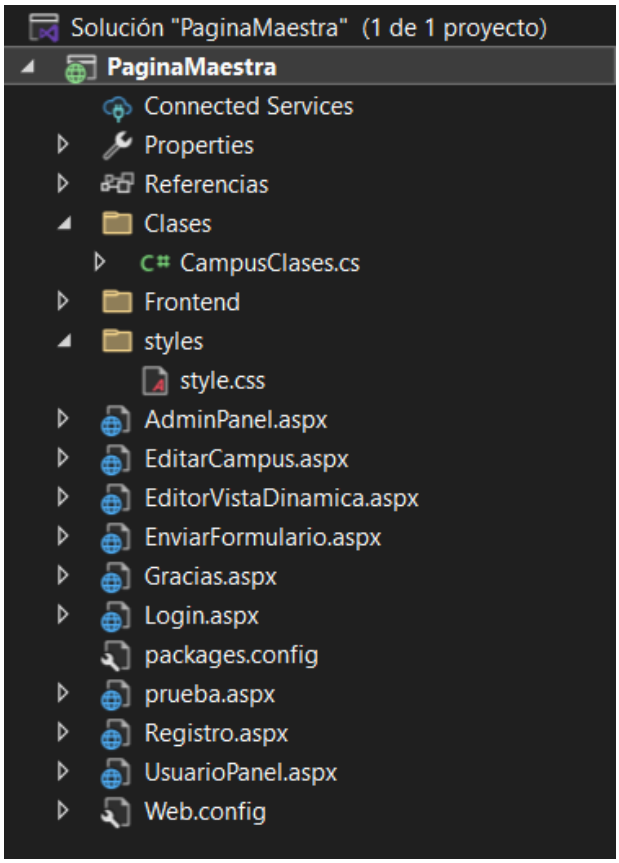


Figura 3. 1 Estructura del Proyecto (Backend)

3.2. DESARROLLO DEL FRONTEND (Vista del sitio web)

El Frontend del sitio web se desarrolló con el objetivo principal de crear una interfaz única y adaptable, con la capacidad de mostrar dinámicamente el contenido de cada uno de los cinco campus disponibles. Para garantizar la compatibilidad y accesibilidad, se emplearon tecnologías web estándar, lo que permite una experiencia de usuario consistente en diversos dispositivos y navegadores.

3.2.1 Estructura del Documento HTML

La estructura base del sitio se define en el archivo **index.html**, que actúa como la plantilla principal. Este documento está organizado en secciones lógicas que corresponden a las diferentes partes de la interfaz, desde la cabecera hasta el pie de página.

Estructura General del Documento

El archivo inicia con la declaración del tipo de documento (<!DOCTYPE html>) y la apertura de la etiqueta <html> con el atributo lang="en", que define el idioma del contenido para los navegadores y motores de búsqueda.

Dentro de la sección <head>, se incluyen:

- Metadatos: como el charset UTF-8 y la viewport para adaptar el diseño a dispositivos móviles.
- Hojas de estilo personalizadas (CSS) organizadas por secciones del sitio (styleHeaderMenu.css, styleCampusInfo.css, etc.).
- Frameworks externos como Bootstrap y FontAwesome, que ayudan con el diseño responsivo y el uso de íconos respectivamente.
- Librerías de animación como AOS para efectos visuales.

Cuerpo del Sitio Web (<body>)

El <body> está dividido en secciones bien identificadas que representan las diferentes partes del sitio web. Cada sección está compuesta por un contenedor con clases personalizadas y, en algunos casos, con identificadores únicos para ser manipulados dinámicamente por JavaScript.

Las secciones principales son:

a) Header y Navegación

```
    <!-- HEADER-MENU -->
    <header>
        <nav class="navbar navbar-expand-lg cabeza_menu">
            <!-- Logos -->
            <div class="logo_menu d-flex align-items-center"></div>

            <!-- Botón Hamburguesa -->
            <button class="navbar-toggler" type="button" data-bs-
toggle="offcanvas" data-bs-target="#menuOffcanvas"
                aria-controls="menuOffcanvas">
                <i class="fas fa-bars"></i>
            </button>


            <!-- Contenido visible solo en escritorio -->
            <div class="menu_contenido d-none d-lg-flex">
                <div id="menuEscritorio" class="btns_menu"></div>
                <div id="redesEscritorio" class="redes_menu"></div>
            </div>
        </nav>
    </header>

    <!-- Offcanvas del menú lateral (Bootstrap) -->
    <div class="offcanvas offcanvas-start" tabindex="-1" id="menuOffcanvas"
aria-labelledby="menuOffcanvasLabel">
        <div class="offcanvas-header">
            <h5 id="menuOffcanvasLabel" class="text-dark"></h5>
            <button type="button" class="btn-close" data-bs-
dismiss="offcanvas" aria-label="Cerrar"></button>
```

```
        </div>
        <div class="offcanvas-body">
            <div id="menuOffcanvasContent" class="btns_menu"></div>
            <div id="redesOffcanvasContent" class="redes_menu"></div>
        </div>
    </div>

    <!-- Contenedor del mega menú y submenús para escritorio -->
    <div id="megaMenuGlobalContainer"></div>
    <div id="submenuPanel" class="submenu_panel"></div>
```

El encabezado está compuesto por una barra de navegación que incluye el logo institucional, un menú adaptado tanto para dispositivos móviles como de escritorio, y botones de redes sociales. Se implementa el componente offcanvas de Bootstrap para mostrar el menú lateral en dispositivos móviles.

b) Video de Presentación

```
    <!-- SECCION VIDEO SEGUNDO APARTADO  -->
    <section class="video-contenedor">
        <video width="100%" autoplay muted loop id="videoCampus"> </video>
    </section>
```

Esta sección muestra un video institucional que se reproduce automáticamente al cargar el sitio.

c) Información del Campus

```
    <!--SECCION MONTERREY TERCER APARTADO-->
    <section class="campus-contenedor" id="infoCampus"></section>

    <!-- IMÁGENES BOTONES COMUNIDAD, REVISTA, OCCCS -->
    <div class="img-ccro"></div>

    <!-- SECCION OFERTA ACADEMICA QUINTO APARTADO-->
    <section class="oferta-contenedor container-fluid"
id="ofertaAcademica"></section>

    <!-- CONTENEDOR DE BOTONES CIRCULARES -->
    <div class="container-btnbajos img-ccro"></div>

    <!-- SECCION SIGUENOS SEPTIMO APARTADO-->
    <section class="siguenos-contenedor"></section>

    <!-- SECCIÓN PIE INFO OCTAVO APARTADO -->
    <footer class="pie_contenedor container py-5" id="footer"></footer>
```

Estos son contenedores dinámicos que cargan información específica del campus seleccionado mediante JavaScript y un archivo JSON externo.

e) Formulario de Contacto

```
<!-- SECCIÓN FORMULARIO ACADÉMICA SEXTO APARTADO -->
<section id="contacto" class="seccion_formulario py-4"> ... </section>
```

Una sección importante donde los usuarios pueden solicitar información. Contiene un formulario con campos validados, un sistema de protección CAPTCHA y conexión con un backend en ASP.NET Web Forms para el envío de datos.

Consideraciones Técnicas

- Se emplean clases personalizadas para tener un control detallado del diseño.
- El uso de Bootstrap 5 permite adaptar la estructura a distintos tamaños de pantalla sin esfuerzos adicionales.
- Los contenedores con IDs (id="infoCampus", id="ofertaAcademica", etc.) están pensados para ser poblados dinámicamente con contenido obtenido desde un archivo datos.json, facilitando así la reutilización del mismo index.html para los diferentes campus.

3.2.2 Estilización con CSS

La estilización con Hojas de Estilo en Cascada (CSS) es fundamental para controlar la apariencia y el diseño de los elementos HTML. A diferencia del HTML que estructura el contenido, el CSS define cómo se verá ese contenido, incluyendo colores, fuentes, tamaños, márgenes, y la disposición en la página. Los estilos se aplican a través de selectores, que identifican los elementos HTML a los que se les aplicarán las reglas de estilo, y propiedades, que son las características que se van a modificar. Esto mejora la mantenibilidad, evita conflictos de estilos y permite una personalización más sencilla.

Enlace de CSS a HTML

Para que los estilos se apliquen, es necesario vincular los archivos CSS al documento HTML. Esto se hace dentro de la sección <head> del HTML, utilizando la etiqueta <link>. Cada archivo CSS que subiste (como styleCampus.css o styleFooter.css) se enlazaría de la siguiente manera:

```
<head>
    <!-- Estilos -->
    <link rel="stylesheet" href="style/styleBotonesFlotantes.css">
    <link rel="stylesheet" href="style/styleHeaderMenu.css">
    <link rel="stylesheet" href="style/styleCampusInfo.css">
    <link rel="stylesheet" href="style/styleComunidad.css">
```

```
    <link rel="stylesheet" href="style/styleOfertaAcademica.css">
    <link rel="stylesheet" href="style/styleEventosbtnBajos.css">
    <link rel="stylesheet" href="style/styleFormulario.css">
    <link rel="stylesheet" href="style/styleCampus.css">
    <link rel="stylesheet" href="style/styleSiguenos.css">
    <link rel="stylesheet" href="style/styleFooter.css">
    <!-- Bootstrap CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
    <!-- Bootstrap JS Bundle (Popper + Bootstrap JS) -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.j
s"></script>
    <!-- FontAwesome para íconos -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.2/css/all.min.css">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.10.5/font/bootstrap-icons.css">
    <!-- AOS CSS -->
    <link href="https://cdn.jsdelivr.net/npm/aos@2.3.4/dist/aos.css"
rel="stylesheet">
</head>
```

La propiedad **href** indica la ruta al archivo CSS, permitiendo al navegador aplicar las reglas de estilo a los elementos correspondientes en el cuerpo del documento.

Estructura de las Hojas de Estilo

Las hojas de estilo que proporcionaste (styleBotonesFlotantes.css, styleCampus.css, etc.) siguen una estructura clara:

Selectores: Los selectores son la forma en que CSS "selecciona" los elementos HTML para aplicarles estilos. Pueden ser:

- Selectores de etiqueta: Seleccionan todos los elementos de un tipo específico (ej. body, h1).
    - Selectores de clase: Seleccionan elementos con un atributo class específico (ej. .seccion-principal).
    - Selectores de ID: Seleccionan un elemento único con un atributo id (ej. #encabezado-sitio).
- Declaraciones de estilo: Dentro de cada selector, se definen las propiedades y sus valores. La sintaxis es propiedad: valor;.

Ejemplo de estructura CSS:

```
/* Selector de clase del archivo styleCampus.css */
.seccion-principal {
```

```
  /* Propiedades y valores */
  margin-top: 100px;
  background-color: #f8f8f8;
  padding: 20px;
}

/* Selector de clase del archivo styleHeaderMenu.css */
.menu-desplegable {
  display: none; /* Propiedad para ocultar el menú inicialmente */
  position: absolute;
  background-color: white;
  z-index: 1000;
}

/* Selector de ID del archivo styleFooter.css */
#pie-de-pagina {
  background-color: #333;
  color: white;
  text-align: center;
  padding: 20px 0;
}
```

Organización de los Estilos

Las hojas de estilo se encuentran en una carpeta llamada style/ y están divididas temáticamente. Cada una se enfoca en una sección del sitio, por ejemplo:

- **styleHeaderMenu.css:** estilos del encabezado y menú principal.
- **styleFormulario.css:** diseño y responsividad del formulario de contacto.
- **styleCampus.css:** disposición visual de los campus y sus tarjetas.
- **styleFooter.css:** diseño del pie de página.
- **styleBotonesFlotantes.css:** botones sociales flotantes.
- Entre otras.

Este enfoque modular permite que cada componente tenga un estilo independiente sin depender de una sola hoja global.

Diseño Responsivo

Para asegurar una buena experiencia de usuario en distintos dispositivos (computadoras, tabletas y móviles), se utilizaron técnicas de diseño responsivo:

- **Media queries:** se aplicaron para ajustar el tamaño de fuentes, distribución de columnas y visibilidad de elementos en diferentes tamaños de pantalla.
- **Bootstrap 5:** se utilizó como framework de diseño, lo cual proporcionó clases predefinidas (container-fluid, col-md-6, d-none d-lg-flex, etc.) que facilitan la adaptación automática del contenido.

Ejemplo de clase utilizada en el formulario para responsividad:

```
<div class="form-cuadro col-12 col-lg-7" data-aos="fade-right" data-aos-
delay="200">
```

Esta línea permite que el formulario ocupe el 100% del ancho en pantallas pequeñas (col-12) y 7/12 partes del ancho en pantallas grandes (col-lg-7).

Diseño Visual y Estética

Se aplicó una línea visual moderna basada en:

- Tipografía legible y jerarquía clara entre títulos y párrafos.
- Colores institucionales, definidos en cada hoja de estilo, para mantener coherencia con la imagen de EP de México.
- Animaciones suaves con la librería AOS (Animate On Scroll), para dar dinamismo al cargar secciones:

```
<section id="contacto" class="seccion_formulario py-4" data-aos="fade-up" data-
aos-duration="1000">
```

- Sombras y bordes redondeados: utilizados en tarjetas, botones y formularios para lograr una estética moderna y profesional.

Componentes con Interacción Visual

Algunos elementos visuales clave fueron diseñados para atraer la atención del usuario y facilitar la navegación:

- Botones flotantes (con íconos de redes sociales).
- Efectos de hover en los campus (cambio de color o animación al pasar el mouse).
- Transiciones animadas entre secciones, ayudadas por AOS y estilos CSS personalizados.

En resumen, el uso combinado de hojas de estilo personalizadas, Bootstrap y buenas prácticas en diseño responsivo permitió desarrollar un sitio web visualmente atractivo, coherente con la identidad institucional y accesible desde cualquier dispositivo.

3.2.3 Comportamiento con JavaScript

El comportamiento dinámico del sitio web se implementó mediante código JavaScript personalizado, cuyo objetivo principal es cargar contenido desde un archivo JSON externo y adaptarlo al campus seleccionado por el usuario, evitando así la necesidad de crear múltiples versiones del mismo sitio.

Carga Dinámica de Datos desde JSON

El flujo general de trabajo para obtener y mostrar la información es el siguiente:

1. Identificación del campus:

Se extrae de la URL el parámetro campus para determinar qué información debe cargarse.

Ejemplo: **index.html?campus=Monterrey**.

```
// 1. Obtener el valor de 'campus' de la URL
function obtenerNombreCampus() {
    const params = new URLSearchParams(window.location.search);
    return params.get("campus");
}
```

2. Petición al archivo JSON:

Mediante fetch(), se obtiene el contenido del archivo datos.json que contiene la información centralizada de todos los campus.

```
// 2. Cargar el archivo JSON
function cargarDatosJSON(url) {
    return fetch(url).then(response => response.json());
}
```

3. Filtrado y visualización de datos:

Una vez cargado el JSON, se localiza el campus correspondiente y se llenan dinámicamente los elementos HTML, como el menú, categorías y programas académicos.

Ejemplo de llenado del select de categorías:

```
categorias.forEach((categoria) => {
    const option = document.createElement("option");
    option.value = categoria.nombre;
    option.textContent = categoria.nombre;
    selectCategoria.appendChild(option);
});
```

4. Cambio dinámico de programas según la categoría seleccionada:

Al seleccionar una categoría, se actualiza automáticamente el listado de programas en el <select id="programa">.

```
selectCategoria.addEventListener("change", () => {
    const nombreCategoriaSeleccionada = selectCategoria.value;
    const categoriaSeleccionada = categorias.find(cat => cat.nombre ===
nombreCategoriaSeleccionada);
    selectPrograma.innerHTML = '<option value="" disabled
selected>Seleccione un programa</option>';
```

```
    if (categoriaSeleccionada?.programas) {
        categoriaSeleccionada.programas.forEach((programa) => {
            const opt = document.createElement("option");
            opt.value = programa.nombre;
            opt.textContent = programa.nombre;
            selectPrograma.appendChild(opt);
        });
    }
});
```

Validación del Formulario

Se implementó una función valida() que comprueba que todos los campos estén completos antes de enviar el formulario:

- ✓ Verifica que el nombre, correo, mensaje, categoría y programa estén seleccionados.
- ✓ Comprueba que el usuario haya pasado el reCAPTCHA.
- ✓ Añade una medida antispam, evitando envíos instantáneos (menos de 3 segundos desde la carga de la página).

```
if (!nombre || !email || !mensaje) {
    alert("Por favor llena todos los campos obligatorios.");
    return false;
}
if (captcha.length === 0) {
    alert("Por favor verifica que no eres un robot.");
    return false;
}
```

Integración con Librerías y Efectos

Además del manejo de datos, el JavaScript activa la librería AOS para animaciones al hacer scroll:

```
    <script>
        AOS.init();
    </script>
```

Esto agrega transiciones visuales en elementos como el formulario, las imágenes y los apartados de campus.

Beneficios de la Implementación

- Escalabilidad: agregar un nuevo campus solo requiere añadir su información al archivo JSON.

- Mantenibilidad: no es necesario modificar el HTML para cambiar la información.
- Interactividad: el contenido se adapta dinámicamente a las acciones del usuario.
- Optimización: se reduce la duplicidad de código HTML y CSS.

3.3 GESTIÓN DEL CONTENIDO ARCHIVO JSON

Para unificar y centralizar la información de los cinco campus en un solo sitio web, se implementó un archivo JSON como fuente principal de datos. Este archivo contiene toda la información que el frontend necesita para mostrar el contenido dinámico según el campus seleccionado.

Estructura General del Archivo JSON

El archivo datos.json está organizado de forma jerárquica, agrupando la información por campus. Cada campus contiene datos como:

- Menú de navegación y submenús.
- Imágenes y banners institucionales.
- Categorías académicas y programas ofrecidos.
- Enlaces a redes sociales.

Ejemplo simplificado de la estructura:

```
{
  "campus": {
    "Monterrey": {
      "titulo": "EP de Monterrey",
      "botonesFlotantes": [
        {
          "imagen": "Próximos Eventos",
          "enlace": "/vistas/vista.html?campus=Monterrey&vista=Entérate y
Eventos"
        },
        {
          "imagen": "WhatsApp",
          "enlace": "https://wa.me/8112142744"
        }
      ],
      "logos": [
        "fotos/Monterrey/logo01.png",
        "fotos/Monterrey/logo02.png",
        "fotos/Monterrey/logo03.png"
      ],
      "horarioAtencion": {
        "lunesViernes": "9:00 a 17:30 hrs",
```

```
      "sabado": "9:00 a 13:00 hrs",
      "informes": "222 948 0240, Extensión 602 & 606",
      "soporte": [
        "222 948 0240",
        "Extensión 642 & 643 & 644"
      ],
      "diplomados": "814 246 6826"
    },
    "menu": [
      {
        "titulo": "CONÓCENOS",
        "opciones": [
          {
            "titulo": "Bienvenida",
            "enlace": "/vistas/vista.html",
            "claveVista": "bienvenida",
            "contenido": {
              "imagenBanner": "fotos/Monterrey/bienvenida-del-rector.jpg",
              "mensajeRector": {
                "titulo": "Estimados:",
              }
            }
          }
      },
      {
        "titulo": "CONTACTOS"
      }
    ],
    "redes_sociales": [
      {
        "nombre": "Facebook",
        "url": "https://www.facebook.com/EPdeMexicoMTY/"
      },
      {
        "nombre": "Instagram",
        "url": "https://www.instagram.com/epdemexicocentro"
      },
      {
        "nombre": "TikTok",
        "url": "https://www.tiktok.com/@ep_de_mexico"
      }
    ],
```

Proceso de Carga y Uso del JSON

1. **Lectura del archivo**
   Se utiliza fetch() para obtener el archivo JSON desde el servidor:

```
cargarDatosJSON("http://localhost:50429/Frontend/data/datos.json")
    .then(data => { /* procesamiento */ });
```

2. **Identificación del campus**
   A través del parámetro en la URL (?campus=Monterrey), el sistema selecciona la sección correspondiente dentro del JSON.

3. **Llenado de elementos HTML**
   Con los datos obtenidos, se rellenan dinámicamente:

   - El menú de navegación.
   - Las categorías y programas académicos.
   - Los textos y enlaces de contacto.
   - Las imágenes y banners de cada sección.

4. **Actualización reactiva**
   Si el usuario selecciona una categoría en el formulario, se filtran los programas asociados usando la información cargada desde el JSON sin necesidad de recargar la página.

Ventajas del Uso de JSON

- ❖ Centralización: todos los datos están en un solo archivo, facilitando su administración.
- ❖ Escalabilidad: se puede agregar un nuevo campus o modificar datos sin tocar el HTML ni el JavaScript principal.
- ❖ Flexibilidad: el mismo index.html se adapta automáticamente a cualquier campus.
- ❖ Compatibilidad: JSON es un formato estándar, soportado por la mayoría de los lenguajes y fácil de integrar con APIs.



Figura 3. 3 Página principal del Campus Monterrey creada con HTML, CSS y JAVASCRIPT con la carga de la información dinámica de un JSON

Con esta estrategia, el sistema no solo resuelve el problema de la duplicidad de sitios, sino que también optimiza el mantenimiento y asegura coherencia visual y de contenido en todos los campus. Te muestro los resultados obtenidos, observa la figura 3.3.

3.4 DESARROLLO DEL PANEL DE ADMINISTRACIÓN (Back-End)

El panel de administración fue desarrollado en ASP.NET Web Forms con el propósito de permitir que el personal autorizado pueda editar el archivo JSON que contiene la información de todos los campus, sin necesidad de conocimientos técnicos en programación.

Este módulo asegura que los cambios realizados en el contenido del sitio web se reflejen de forma inmediata y coherente en todas las secciones del frontend.

3.4.1 Objetivo del Panel

El objetivo de este módulo es evitar la edición manual del archivo JSON, reduciendo riesgos de errores de formato y permitiendo que personal no técnico pueda actualizar la información institucional de forma segura y estructurada.

3.4.2 Estructura General

El panel está compuesto por dos elementos principales:

1. **Interfaz gráfica (EditarCampus.aspx)**
   - Contiene controles como DropDownList, TextBox, Button y GridView para mostrar y editar los datos.
   - Está diseñada para mostrar las categorías y programas asociados a un campus específico.
   - Los elementos se organizan en secciones: selección de campus, lista de categorías y formulario de edición/agregado.
2. **Lógica de negocio (EditarCampus.aspx.cs)**
   - Carga el archivo datos.json en memoria.
   - Mapea los datos JSON a objetos C#.
   - Controla las acciones de los botones (agregar, modificar, eliminar).
   - Vuelve a serializar el JSON y guarda los cambios en el servidor.

Observa la figura 3.4 de como se ve la interfaz de edición.

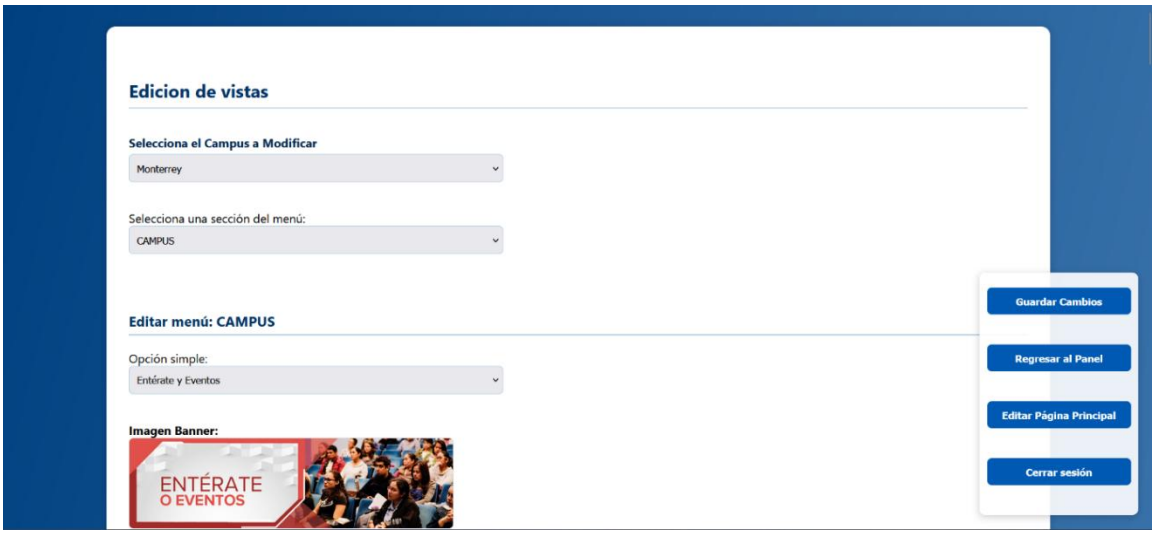


Figura 3. 4 Interfaz de administrador para editar la información del campus seleccionado en el Json

3.4.3 Flujo de Funcionamiento

1. **Carga inicial del archivo JSON**
   Cuando la página se carga por primera vez, el código en Page_Load lee el archivo datos.json:

```
private string RutaJson => Server.MapPath("~/Frontend/data/datos.json");
private static CampusData datos;
private static string campusActual;
```

   Aquí, DatosCampus es una clase modelo que refleja la estructura del JSON, facilitando su manipulación.

2. **Selección de campus**
   - Un DropDownList permite seleccionar el campus que se desea editar.
   - Al cambiar de campus, se cargan dinámicamente las categorías y programas asociados a ese campus.
3. **Edición de datos**
   - Los datos actuales se muestran en GridView o ListBox.
   - El administrador puede seleccionar un elemento para editarlo, modificando su nombre, descripción o lista de programas.
4. **Agregado de nueva información**
   - Un formulario permite introducir nuevos programas o categorías.
   - El sistema añade el nuevo objeto a la estructura en memoria.
5. **Guardado de cambios**

Una vez que se aplican modificaciones, el código convierte nuevamente los objetos C# a JSON y sobrescribe el archivo original:

```
File.WriteAllText( RutaJson, JsonConvert.SerializeObject(
datos,Formatting.Indented, new JsonSerializerSettings { NullValueHandling =
NullValueHandling.Ignore }));
```

3.4.4 Validaciones Implementadas

El panel incorpora validaciones para asegurar la integridad de los datos:

- Evitar que se agreguen categorías o programas con nombres duplicados.
- Comprobar que los campos requeridos no estén vacíos.
- Verificar que la estructura del JSON resultante sea válida antes de guardarla.
- Confirmar con el usuario acciones críticas como eliminaciones.

3.4.5 Seguridad

- La seguridad es un pilar fundamental del panel de administración, y se implementa a través de varios mecanismos:

- Autenticación de Usuarios: El archivo Login.aspx maneja el proceso de autenticación. El Login.aspx.cs valida las credenciales (por ejemplo, con un hash seguro) y, si son correctas, crea una cookie de autenticación para la sesión del usuario.
- Control de Acceso: El archivo Web.config define las reglas de autorización, restringiendo el acceso a las carpetas y archivos del panel (/Admin/) a solo los usuarios autenticados.
- Gestión de Cuentas: La página Registro.aspx y su lógica asociada (Registro.aspx.cs) permiten la creación de nuevos usuarios administradores, siguiendo un proceso controlado para evitar accesos no autorizados.
- Validación en el Servidor: La lógica de negocio en los archivos .cs verifica que las acciones sean realizadas por usuarios con los permisos adecuados, impidiendo que se ejecuten operaciones sin la debida autorización.

3.4.6 Beneficios de la Implementación

- Eficiencia: Se eliminó la necesidad de abrir y editar manualmente archivos de configuración, simplificando las tareas de mantenimiento.
- Seguridad y Fiabilidad: Las validaciones, la autenticación y las restricciones de acceso reducen significativamente el riesgo de corrupción de datos y accesos no autorizados.
- Usabilidad: La interfaz gráfica es comprensible y accesible incluso para usuarios sin conocimientos técnicos, democratizando la gestión de contenido
- Escalabilidad: La arquitectura modular y el uso de un archivo Web.config permiten agregar nuevas funcionalidades o campus de manera sencilla sin modificar el código principal.

CONCLUSIÓN

Puedo afirmar que el objetivo principal del proyecto, el desarrollo e implementación de un sitio web unificado para los cinco campus de EP de México, se cumplió al 100%. Sin embargo, la implementación no estuvo exenta de desafíos. En el backend, enfrenté complicaciones al trabajar con controles dinámicos en ASP.NET Web Forms, lo que me llevó a dedicar un tiempo considerable para corregir la funcionalidad en el postback.

El mayor obstáculo surgió al intentar desplegar el sitio, ya que el servidor de la empresa, una instancia de AWS con Ubuntu 22, era incompatible con .NET Framework. Para resolver esta situación, implementé una solución innovadora utilizando Apache para manejar las solicitudes web y Mono, una implementación de .NET para Linux, que me permitió ejecutar la aplicación sin problemas en el entorno del servidor.

La implementación de este proyecto ha traído ventajas significativas para la empresa. Logramos centralizar la información de los cinco campus, lo cual mejoró la imagen institucional, haciéndola más moderna y coherente. El nuevo sitio web ofrece una mejor experiencia de usuario y su administración es más eficiente, ya que el contenido se gestiona de forma dinámica a través de un solo archivo JSON. Considero que el proyecto es adecuado tanto para su uso como para su administración. En cuanto a trabajos futuros, he identificado una oportunidad clave: mejorar la interfaz de administración en el backend. Esto permitirá que la edición del archivo JSON sea aún más sencilla e intuitiva para el personal de la empresa.

Realizar esta estadía práctica me proporcionó un gran aprendizaje. No solo puse en práctica mis conocimientos técnicos, sino que también desarrollé una valiosa habilidad para la resolución de problemas. Aprendí a depurar errores complejos en controles dinámicos y, sobre todo, a adaptarme y encontrar soluciones creativas cuando me enfrenté a una incompatibilidad tecnológica, como fue el caso de usar Mono y Apache para ejecutar una aplicación de .NET Framework en Linux.

Como resultado de mi estadía, la empresa ahora cuenta con una plataforma web unificada y robusta. Este cambio no solo mejora su presencia digital y la comunicación con sus usuarios, sino que también establece una base tecnológica sólida para futuras actualizaciones, proyectando una imagen de innovación y modernidad que antes no tenía.

GLOSARIO

- **Accesibilidad Web**: Principio y conjunto de técnicas que aseguran que un sitio web pueda ser usado por el mayor número de personas posible, incluyendo aquellas con discapacidades.
- **Adaptabilidad (Diseño Web)**: Capacidad de un sitio web para ajustarse correctamente a diferentes dispositivos y tamaños de pantalla.
- **AJAX (Asynchronous JavaScript and XML)**: Técnica que permite actualizar partes de una página web sin recargarla completamente, mejorando la interactividad y rapidez.
- **Ancho de Banda**: Cantidad de datos que se pueden transferir en un período de tiempo determinado a través de una conexión de red.
- **Aplicación Web**: Programa que se ejecuta en un servidor y al que se accede mediante un navegador, ofreciendo funcionalidades dinámicas e interactivas.
- **Back-End**: Parte de una aplicación web que gestiona la lógica, el procesamiento de datos y la conexión con bases de datos, generalmente en el servidor.
- **Base de Datos**: Conjunto estructurado de datos almacenados y gestionados por un sistema que permite su consulta, actualización y administración.
- **Campus Virtual**: Plataforma en línea que permite la educación a distancia mediante recursos digitales y aulas virtuales.
- **CSS (Cascading Style Sheets)**: Lenguaje de hojas de estilo usado para definir el aspecto visual de un sitio web (colores, tipografía, disposición, etc.).
- **Cliente FTP**: Programa que permite transferir archivos entre un ordenador y un servidor mediante el protocolo FTP.
- **Código Fuente**: Conjunto de instrucciones escritas en un lenguaje de programación que definen el funcionamiento de un programa o página web.
- **Comercio Electrónico**: Actividad de compra y venta de bienes o servicios a través de Internet.
- **Contenido Dinámico**: Información que se actualiza o cambia en tiempo real en una página web, en respuesta a la interacción del usuario o a datos del servidor.
- **Diseño Responsivo**: Técnica de diseño web que permite que las páginas se adapten automáticamente a diferentes resoluciones y tamaños de pantalla.
- **Dominio**: Nombre único que identifica un sitio web en Internet, como www.ejemplo.com.
- **Documento HTML**: Archivo que contiene el código estructurado para la creación de páginas web.
- **Drop-down Menu (Menú Desplegable)**: Elemento de navegación que muestra opciones adicionales al interactuar con él.
- **Eficiencia Operativa (Web)**: Optimización de recursos y procesos para mejorar el rendimiento y la funcionalidad de un sitio web.
- **Enlace (Link)**: Elemento que conecta un recurso web con otro, ya sea en el mismo sitio o en otro diferente.

- **Estética Web**: Conjunto de elementos visuales que determinan la apariencia de un sitio web y transmiten su identidad.
- **Extensión de Archivo**: Sufijo al final del nombre de un archivo que indica su formato (por ejemplo, .html, .css, .jpg).
- **Frontend**: Parte de un sitio web con la que interactúa directamente el usuario, desarrollada con tecnologías como HTML, CSS y JavaScript.
- **FTP (File Transfer Protocol)**: Protocolo que permite transferir archivos entre sistemas a través de Internet.
- **Framework**: Conjunto de herramientas y componentes predefinidos que facilitan el desarrollo de software.
- **HTML5 (HyperText Markup Language)**: Versión más reciente del lenguaje de marcado para estructurar contenido en la web.
- **Hosting**: Servicio que proporciona un servidor para almacenar y publicar sitios web en Internet.
- **HTTP / HTTPS**: Protocolos para la transferencia de información en la web; HTTPS incluye un cifrado para mayor seguridad
- **JavaScript**: Lenguaje de programación utilizado para añadir interactividad y dinamismo a las páginas web.
- **JSON (JavaScript Object Notation)**: Formato ligero para el intercambio de datos, fácil de leer para humanos y procesar por máquinas.
- **Metodología de Desarrollo**: Conjunto de pasos y técnicas utilizadas para planificar, crear, probar y mantener un proyecto web.
- **Multiplataforma**: Capacidad de un software para ejecutarse en diferentes sistemas operativos o dispositivos.
- **Navegador Web**: Programa que permite visualizar e interactuar con páginas de Internet (por ejemplo, Chrome, Firefox, Edge).
- **Nombre de Usuario**: Identificador único que utiliza una persona para acceder a un sistema o servicio.
- **Panel de Administración (Back-End)**: Interfaz restringida que permite gestionar el contenido y configuración de un sitio web.
- **Planteamiento del Problema**: Descripción detallada de la situación que motiva el desarrollo de un proyecto.
- **Protocolo**: Conjunto de reglas que determinan cómo se transmiten los datos en una red.
- **Red Social**: Plataforma digital que facilita la comunicación e interacción entre personas o comunidades.
- **Repositorio**: Espacio de almacenamiento donde se guarda y mantiene código o contenido digital.
- **Servidor Web**: Computadora o sistema que almacena y entrega páginas y recursos web a los navegadores.
- **Sitio Web Unificado**: Plataforma que centraliza la información de varias sedes o secciones en un solo dominio.
- **Software Libre**: Programa que puede ser usado, modificado y distribuido libremente.

- **Tesina**: Documento académico que desarrolla un tema de investigación o práctica profesional como requisito para obtener un grado.
- **Tipografía Web**: Selección y uso de fuentes tipográficas en el diseño de sitios web.

BIBLIOGRAFÍAS

- Freeman, A. (2020). HTML5: Guía definitiva (7.ª ed.). Apress.
- Gauchat, J. D. (2018). El gran libro de HTML5, CSS3 y JavaScript (3.ª ed.). Marcombo.
- Goetter, R. (2017). HTML5 y CSS3: Técnicas avanzadas para el diseño de sitios web. ENI Ediciones.
- Jaimez González, C. R. (2015). Programación de web estático. Universidad Autónoma Metropolitana. http://dccd.cua.uam.mx/libros/archivos/05programacion_web_estatico.pdf
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Mateu, C., Megías Jiménez, D., & Mas, J. (2004). Desarrollo de aplicaciones web. Fundació per a la Universitat Oberta de Catalunya.
- McFarland, D. S. (2012). CSS3: El manual que falta. McGraw-Hill Education.
- Meyer, E. A. (2017). CSS: The Definitive Guide. O'Reilly Media.
- Pardo Niebla, M. (2014). CREACIÓN Y DISEÑO WEB. ANAYA MULTIMEDIA.
- Secretaría de Educación Pública. (2011). Elaboración de Páginas Web. Secretaría de Educación Pública.
- Aguirre, S. (2023). ASP.NET: Aplicaciones Web. Editorial RA-MA.
- Duckett, J. (2011). HTML y CSS: Diseño y construcción de sitios web. Wiley.
- Haberbeke, M. (2024). Eloquent JavaScript: A Modern Introduction to Programming. No Starch Press.
- Millán, R. J., & Millán, C. (2008). Páginas web. Creaciones Copyright.
- Subdirección General de Tecnologías de la Información y las Comunicaciones. (2019, 15 de marzo). Normas y estándares para el desarrollo de sitios web. Gobierno de España. https://administracionelectronica.gob.es/pae_Home/dms/pae_Home/documentos/Normas/Normas_y_Estándares_para_el_Desarrollo_de_Sitios_Web.pdf